

Predicción de Costos Médicos

Mejora de predicción con ingeniería de características, modelos penalizados, ensambles y pilas

Índice

1. Introducción	4
2. Diagnóstico y Justificación de la Estrategia de Mejora	4
2.1. Punto de Partida	4
2.2. Relaciones no lineales no capturadas	5
2.3. Selección de la estrategia de mejora	6
2.4. Plan de mejora progresiva	7
3. Ingeniería de características	7
3.1. Diseño de las variables derivadas	7
3.2. Análisis estadístico de las variables derivadas	10
3.3. Validación visual de las interacciones	11
3.4. Impacto en la matriz de correlación	14
3.5. Re-entrenamiento del modelo lineal con variables derivadas	16
3.6. Evaluación del impacto de la ingeniería de características	18
4. Regresiones penalizadas	20
4.1. Introducción y justificación	20
4.2. Preparación de los datos para glmnet	20
4.3. Regresión Ridge	21
4.4. Regresión Lasso	25
4.5. Lasso Adaptativo	29
4.5.1. Estandarización de los datos	29
4.5.2. Ridge sobre datos estandarizados	30
4.5.3. Cálculo de los pesos adaptativos	30
4.5.4. Ajuste del Lasso Adaptativo	31
5. Métodos de ensamble	34
5.1. Random Forest base	34
5.1.1. Importancia de variables	36
5.1.2. Evaluación del modelo base	37
5.1.3. Ajuste de hiperparámetros mediante validación cruzada	39
5.1.4. Evaluación del Random Forest ajustado	40
5.2. XGBoost	43
5.2.1. Preparación de los datos	43
5.2.2. Modelo base	44
5.2.3. Búsqueda de hiperparámetros mediante grid search con validación cruzada	47
5.2.4. Modelo final ajustado	49
6. Modelos de Pila (Stacking)	52
6.1. Introducción y justificación	52
6.2. Preparación del protocolo de validación cruzada compartido	53
6.3. Definición de los espacios de búsqueda de hiperparámetros	53
6.4. Entrenamiento simultáneo de los modelos base	54

6.4.1. Rendimiento individual de los modelos base	55
6.4.2. Análisis de diversidad entre modelos base	56
6.5. Entrenamiento del meta-modelo	56
6.5.1. Análisis de diversidad entre modelos base	56
6.5.2. Meta-modelo Random Forest	57
6.6. Evaluación en el conjunto de prueba y selección del mejor meta-modelo	57
6.7. Análisis del gráfico Real vs Predicho	58
6.8. H2O AutoML	60
6.8.1. Introducción y configuración	60
6.8.2. Configuración del proceso AutoML	61
6.8.3. Limitación técnica ausencia de XGBoost	62
6.8.4. Análisis del leaderboard	62
6.8.5. Evaluación del modelo líder en el conjunto de prueba	63
6.8.6. Análisis del gráfico real vs predicho	63
7. Conclusiones	65
7.1. Análisis comparativo final	65
7.2. Conclusión final	70

1. Introducción

La primera fase de este proyecto estableció la viabilidad de un modelo predictivo para la estimación de costos médicos en el sector asegurador, comparando una *Regresión Lineal Múltiple* y un *Árbol*. Los resultados obtenidos sobre el conjunto de prueba revelaron que el *Árbol* superó al modelo lineal en todas las métricas evaluadas, alcanzando un coeficiente de determinación $R^2 = 0.8667$ y un *Error Absoluto Medio* (MAE) de \$3,251.90 USD.

No obstante, un análisis crítico de estos resultados indica que existe un margen de mejora sustancial. El gráfico de dispersión *Real vs Predicho* del mejor modelo de la parte I mostró una dispersión considerable en los perfiles de alto costo, evidenciando que los algoritmos base no lograron capturar plenamente las interacciones multiplicativas entre variables identificadas durante el análisis exploratorio, en particular, la sinergia entre el tabaquismo y el índice de masa corporal elevado.

Con base en este diagnóstico, el presente reporte documenta la segunda fase del proyecto, estructurada en cuatro etapas de mejora progresiva. Primero, se aplica *Ingeniería de Características* para codificar explícitamente las interacciones no lineales detectadas en la fase exploratoria. Segundo, se introducen *Regresiones Penalizadas* (Ridge, Lasso y Lasso Adaptativo) como puente entre los modelos lineales base y los métodos de ensamble, aprovechando su capacidad de selección automática de variables. Tercero, se entrenan métodos de Ensamble (Random Forest y XGBoost) con ajuste sistemático de hiperparámetros para capturar patrones complejos que los modelos individuales no pueden modelar. Finalmente, se presentan conclusiones cuantitativas sobre la mejora acumulada obtenida a lo largo del proceso.

2. Diagnóstico y Justificación de la Estrategia de Mejora

2.1. Punto de Partida

La primera fase del proyecto establece dos modelos predictivos base evaluados sobre un conjunto de pruebas de 400 observaciones. La *Tabla 1* resume los resultados obtenidos.

<i>Modelo</i>	<i>MAE (USD)</i>	<i>RMSE (USD)</i>	<i>R</i>
Regresión Lineal Múltiple	4,160.66	5,655.73	0.8085
Árbol	3,251.90	4,688.57	0.8667

Tabla 1. Métricas de rendimiento de los modelos base entrenados en la Fase I.

Si bien el árbol fue seleccionado como el modelo óptimo de la fase anterior, un análisis crítico de sus resultados revela limitaciones concretas que justifican una segunda fase de mejora. Un R^2 de 0.8667 implica que el modelo no puede explicar aproximadamente el 13% de la varianza total en los costos médicos. Desde la perspectiva del negocio asegurados, un RMSE de \$4,688.57 USD representa un margen de error financiero significativo que puede traducirse en primas mal calibradas y pérdidas operativas para la empresa.

El análisis visual de los gráficos *Real vs Predicho* generados en la Fase I expuso la causa estructural de este margen de error. La Regresión Lineal, al asumir relaciones puramente aditivas, subestimó sistemáticamente los costos de los pacientes en el segmento de alto riesgo (cargos superiores a \$40,000 USD). Por su parte, el Árbol mejoró esta segmentación, pero su arquitectura limitada a 5 niveles de profundidad produjo predicciones escalonadas que no capturan la variación continua dentro de cada perfil de riesgo.

2.2. Relaciones no lineales no capturadas

El análisis exploratorio bivariado de la fase I documentó un hallazgo crítico, la existencia de una interacción multiplicativa entre el tabaquismo y el índice de masa corporal. Este fenómeno implica que el impacto financiero de la obesidad no es constante, sino que se amplifica exponencialmente cuando converge con el tabaquismo.

Los modelos de la fase I trataron estas variables de forma independiente. En términos matemáticos, ambos algoritmos calculaban el costo como una función aditiva, asumiendo que fumar y tener obesidad simplemente suman sus efectos por separado. Sin embargo, los datos del conjunto de entrenamiento demuestran que esta suposición es incorrecta, un fumador con BMI superior a 30 no cuesta \$23,057 (tabaquismo) más \$364 por punto de BMI (obesidad), sino que incurre en costos que superan los \$42,000 USD, un valor que ninguna función lineal puede reproducir sin codificar explícitamente esta interacción.

La *Figura 1* ilustra este efecto multiplicador, los pacientes fumadores (en rojo) con valores de smoker x BMI superiores a 30 se concentra en la región de costos más extremos del conjunto de datos, formando una nube de puntos con una pendiente notablemente más pronunciada que la del resto de la población.



Figura 1. Interacción smoker x BMI vs Charges

2.3. Selección de la estrategia de mejora

Ante el diagnóstico expuesto, la metodología KDD contempla diversas estrategias para mejorar el rendimiento predictivo. Para este proyecto se evaluaron las siguientes alternativas,

Balanceo de clases y remuestreo, Esta técnica están diseñadas para corregir desequilibrios en la distribución de la variable objetivo en problemas de clasificación. Dado que el presente proyecto aborda un problema de regresión con una variable continua, la aplicación de estas estrategias resulta metodológicamente inapropiada y fue descartada.

Ajuste de hiperparámetros sobre los modelos base, Estas técnicas están diseñadas para corregir desequilibrios en la distribución de la variable objetivo en problemas de clasificación. Dado que el presente proyecto aborda un problema de regresión con una variable continua, la aplicación de estas estrategias resulta metodológicamente inapropiada y fue descartada.

Ingeniería de características, Esta estrategia aborda directamente la causa de raíz del error. Al construir nuevas variables que codifican explícitamente las interacciones multiplicativas detectadas en el EDA, se les proporciona al modelo información que antes estaba implícita en

los datos pero que ningún algoritmo podía extraer de forma autónoma con las variables originales.

En consecuencia, se determinó que la *Ingeniería de Características* constituye la intervención prioritaria de esta segunda fase, dado que ataca directamente la limitación estructural identificada y beneficia simultáneamente a todos los algoritmos que se entrenarán posteriormente, tanto los modelos penalizados como los métodos de ensamble.

2.4. Plan de mejora progresiva

La estrategia de mejora se estructuró en cuatro etapas secuenciales, donde cada fase se construye sobre los resultados de la anterior,

Etapla 1 Ingeniería de características: Construcción de cinco nuevas variables derivadas que codifican las relaciones no lineales e interacciones identificadas en el EDA. Se re-entrenará la regresión lineal sobre el espacio de características ampliado para cuantificar la ganancia atribuible exclusivamente a esta intervención.

Etapla 2 Regresiones penalizadas: Aplicación de Ridge, Lasso y Lasso Adaptativo sobre el conjunto de características enriquecido. Estos modelos no solo incorporan regularización para controlar el sobreajuste derivado del mayor número de variables, sino que realizan selección automática de características, identificando cuáles de las variables nuevas aportan información real y cuáles son redundantes.

Etapla 3 Métodos de ensamble: Entrenamiento de Random Forest y XGBoost con ajuste sistemático de hiperparámetros mediante validación cruzada de 10 pliegues. Estos algoritmos tienen la capacidad inherente de modelar relaciones no lineales complejas y representan el límite superior de rendimiento predictivo alcanzable con este conjunto de datos.

Etapla 4 Métodos de pila: Implementación de ensambles de segundo nivel para combinar las predicciones de familias algorítmicas distintas (modelos lineales, árboles y regresiones penalizadas). Mediante el uso de meta-modelos supervisados (paquete caret) y la exploración con una plataforma de aprendizaje automatizado (H2O AutoML), esta fase busca aprovechar los errores complementarios de cada algoritmo base, aprendiendo a ponderar sus estimaciones para intentar superar el techo de rendimiento establecido por los modelos individuales.

Etapla 5 Evaluación comparativa final: Consolidación de todas las métricas en una tabla comparativa única y selección del modelo óptimo con base en criterios cuantitativos y de interpretabilidad para el negocio asegurador.

3. Ingeniería de características

3.1. Diseño de las variables derivadas

El diagnóstico presentado en la sección anterior determinó que la causa principal del error en los modelos base fue la incapacidad de capturar relaciones multiplicativas entre variables. Para abordar esta limitación de forma directa, se aplicó una técnica de ingeniería de características, la cual consiste en construir nuevas variables a partir de las existentes para codificar explícitamente la información que los algoritmos no podían extraer de forma autónoma.

Como primer paso, se verificó el estado del conjunto de datos de esta fase.

```
> dim(insurance_data) [1]
1337  14
> str(insurance_data)
'data.frame', 1337 obs. of  14 variables,
 $ age           , int  19 18 28 33 32 31 46 37 37 60 ...
 $ sex           , num  1 0 0 0 0 1 1 1 0 1 ...
 $ bmi           , num  27.9 33.8 33 22.7 28.9 ...
 $ children      , int  0 1 3 0 0 0 1 3 2 0 ...
 $ smoker        , num  1 0 0 0 0 0 0 0 0 0 ...
 $ charges       , num  16885 1726 4449 21984 3867 ...
 $ region_northeast, num  0 0 0 0 0 0 0 0 1 0 ...
 $ region_northwest, num  0 0 0 1 1 0 0 1 0 1 ...
 $ region_southeast, num  0 1 1 0 0 1 1 0 0 0 ...
 $ age2          , num  361 324 784 1089 1024 ...
 $ smoker_bmi    , num  27.9 0 0 0 0 0 0 0 0 0 ...
 $ smoker_age    , num  19 0 0 0 0 0 0 0 0 0 ...
 $ bmi_obese     , num  0 1 1 0 0 0 1 0 0 0 ...
 $ smoker_obese  , num  0 0 0 0 0 0 0 0 0 0 ...
```

Las consulta de dimensiones confirma que el conjunto de datos cuenta con 1,337 observaciones y 9 variables, todas en formato numérico (int o num). Esto es relevante porque los algoritmos de regresión operan exclusivamente sobre espacios vectoriales numéricos y la ausencia de variables de texto confirma que no se requiere ninguna codificación adicional antes de proceder al modelado.

A continuación se construyeron cinco nuevas variables mediante la función *mutate()*

```
> insurance_data <- insurance_data %>%
+   mutate(
+     age2          = age^2,
+     smoker_bmi    = smoker * bmi,
+     smoker_age    = smoker * age,
+     bmi_obese     = ifelse(bmi >= 30, 1, 0),
+     smoker_obese  = smoker * bmi_obese
+   )
```

El diseño de cada variable responde a una hipótesis clínica y actuarial específica, **age2**, La relación entre la edad y los costos médicos no es lineal. El deterioro de la salud no avanza al mismo ritmo durante toda la vida, los costos médicos tienden a acelerar conforme el paciente envejece. Elevar la edad al cuadrado permite que el modelo capture este crecimiento progresivo y diferenciado, reconociendo que el salto de costos entre los 60 y 64 años es cualitativamente distinto al que ocurre entre los 22 y los 26.

smoker_bmi, Esta variable codifica la sinergia multiplicativa entre el tabaquismo y el índice de masa corporal. Al multiplicar ambas variables, se le proporciona al modelo información que estaba implícita en los datos pero era invisible con las variables originales

separadas, el impacto financiero del BMI no es constante, sino que se amplifica de forma significativa cuando el asegurado es fumador.

smoker_age, Con la misma lógica, esta variable captura la interacción entre el tabaquismo y la edad. Un asegurado fumador de edad avanzada no solo acumula el riesgo del envejecimiento y el riesgo del tabaquismo de forma independiente, sino que ambos factores se potencian mutuamente, generando un perfil de costo superior al que cualquier modelo aditivo podría estimar.

bmi_obese, Variable indicadora binaria que clasifica a cada asegurado según el umbral de obesidad clínica establecido por la Organización Mundial de la Salud ($BMI \geq 30$). Un valor de 1 identifica a individuos con obesidad y un valor de 0 a aquellos con peso normal o sobrepeso sin obesidad.

smoker_obese, Esta variable identifica el perfil de máximo riesgo financiero del dataset, el asegurado que simultáneamente fuma y padece obesidad clínica. Al ser el producto de *smoker* y *bmi_obese*, toma el valor de 1 únicamente cuando ambas condiciones se presenta al mismo tiempo, permitiendo al modelo asignar un peso predictivo específico y diferenciado a esta combinación crítica.

Para verificar la correcta creación de las variables, se inspeccionaron los primeros diez registros del conjunto de datos

```
> insurance_data %>%
+   select(age, age2, bmi, bmi_obese, smoker,
+          smoker_bmi, smoker_age, smoker_obese) %>%
+   head(10)
   age age2    bmi bmi_obese smoker smoker_bmi
smoker_age smoker_obese
1   19  361 27.900         0      1      27.9      19
0
2   18  324 33.770         1      0       0.0       0
0
3   28  784 33.000         1      0       0.0       0
0
4   33 1089 22.705         0      0       0.0       0
0
5   32 1024 28.880         0      0       0.0       0
0
6   31  961 25.740         0      0       0.0       0
0
7   46 2116 33.440         1      0       0.0       0
0
8   37 1369 27.740         0      0       0.0       0
0
9   37 1369 29.830         0      0       0.0       0
0
10  60 3600 25.840         0      0       0.0       0
0
```

La verificación confirma que la lógica matemática funciona correctamente. El registro 1 corresponde a un asegurado de 19 años que si fuma con un BMI de 27.9, por debajo del umbral de obesidad. Por ello, *smoker_bmi* es 0, ya que aunque fuma, no cumple el criterio de obesidad. Los registros del 2 al 10 corresponden a no fumadores, razón por la cual todas sus variables de interacción valen exactamente cero, independientemente de su BMI o edad.

3.2. Análisis estadístico de las variables derivadas

Con las nuevas variables correctamente construidas, se extrajo su estadística descriptiva para validar su distribución y comportamiento

```
> insurance_data %>%
+   select(age2, smoker_bmi, smoker_age, bmi_obese, smoker_obese)
%>%
+   summary()
```

	age2	smoker_bmi	smoker_age
bmi_obese	smoker_obese		
Min. , 324	Min. , 0.000	Min. , 0.000	Min. , 0.000
Min. , 0.0000			
1st Qu., 729	1st Qu., 0.000	1st Qu., 0.000	1st Qu., 0.000
1st Qu., 0.0000			
Median , 1521	Median , 0.000	Median , 0.000	Median , 1.000
Median , 0.0000			
Mean , 1735	Mean , 6.293	Mean , 7.893	Mean , 0.528
Mean , 0.1085			
3rd Qu., 2601	3rd Qu., 0.000	3rd Qu., 0.000	3rd Qu., 1.000
3rd Qu., 0.0000			
Max. , 4096	Max. , 52.580	Max. , 64.000	Max. , 1.000
Max. , 1.0000			

El análisis de cada variable derivada revela comportamientos consistentes con las hipótesis de diseño, **age2**, El valor mínimo de 324 corresponde exactamente a 18^2 y el máximo de 4,096 a 64^2 , confirmando que la transformación respeta el rango original de edades. La mediana de 1,521 equivale aproximadamente a 39^2 , valor coherente con la mediana de edad reportada en la fase I. La ligera superioridad de la media (1,735) sobre la mediana refleja la asimetría positiva original de la variable edad.

smoker_bmi y smoker_age, En ambas variables, el primer, segundo y tercer cuartil valen cero, lo que indica que al menos el 75% de las observaciones son no fumadores. Al multiplicar por cero ($smoker = 0$), el producto siempre es cero, independientemente del BMI o la edad. La media de 6.293 en *smoker_bmi* y de 7.893 en *smoker_age* refleja exclusivamente el comportamiento del 20% fumador de la muestra. El máximo de 52.58 en *smoker_bmi*

corresponde al asegurado fumador con el BMI más alto del dataset, mientras que el máximo de 64 en *smoker_age* identifica al fumador de mayor edad registrado.

bmi_obese, Esta variable presenta el hallazgo más revelador desde la perspectiva de salud pública. A diferencia de las variables de interacción, su mediana vale 1, lo que implica que más del 50% de los asegurados supera el umbral de obesidad clínica. La media de 0.528 confirma que el 52.8% de la cartera de clientes padece obesidad, reafirmando el hallazgo identificado en el análisis exploratorio de la fase I y subrayando su relevancia como factor de riesgo estructural para la aseguradora.

smoker_obese, Con una media de 0.1085 y los tres cuartiles en cero, esta variable identifica al subgrupo de máximo riesgo del dataset. Únicamente del 10.85% de los asegurados pertenece a este perfil crítico, definido por la presencia simultánea de tabaquismo y obesidad clínica. Aunque este segmento es marginal en tamaño, concentra los costos más extremos del conjunto de datos, como se verificará en el análisis visual de la sección siguiente.

3.3. Validación visual de las interacciones

Para confirmar empíricamente que las nuevas variables capturan señal predictiva real, se generaron dos gráficos de dispersión que evalúan visualmente el impacto de las interacciones construidas.

```
> ggplot(insurance_data, aes(x = smoker_bmi, y = charges,
+                             color = factor(smoker))) +
+   geom_point(alpha = 0.4) +
+   labs(title = "Interacción smoker×BMI vs Charges",
+         x = "smoker × BMI", y = "Charges (USD)",
+         color = "Fumador") +
+   scale_color_manual(values = c("0" = "#0073C2FF", "1" =
+ "#CD534CFF"))
```

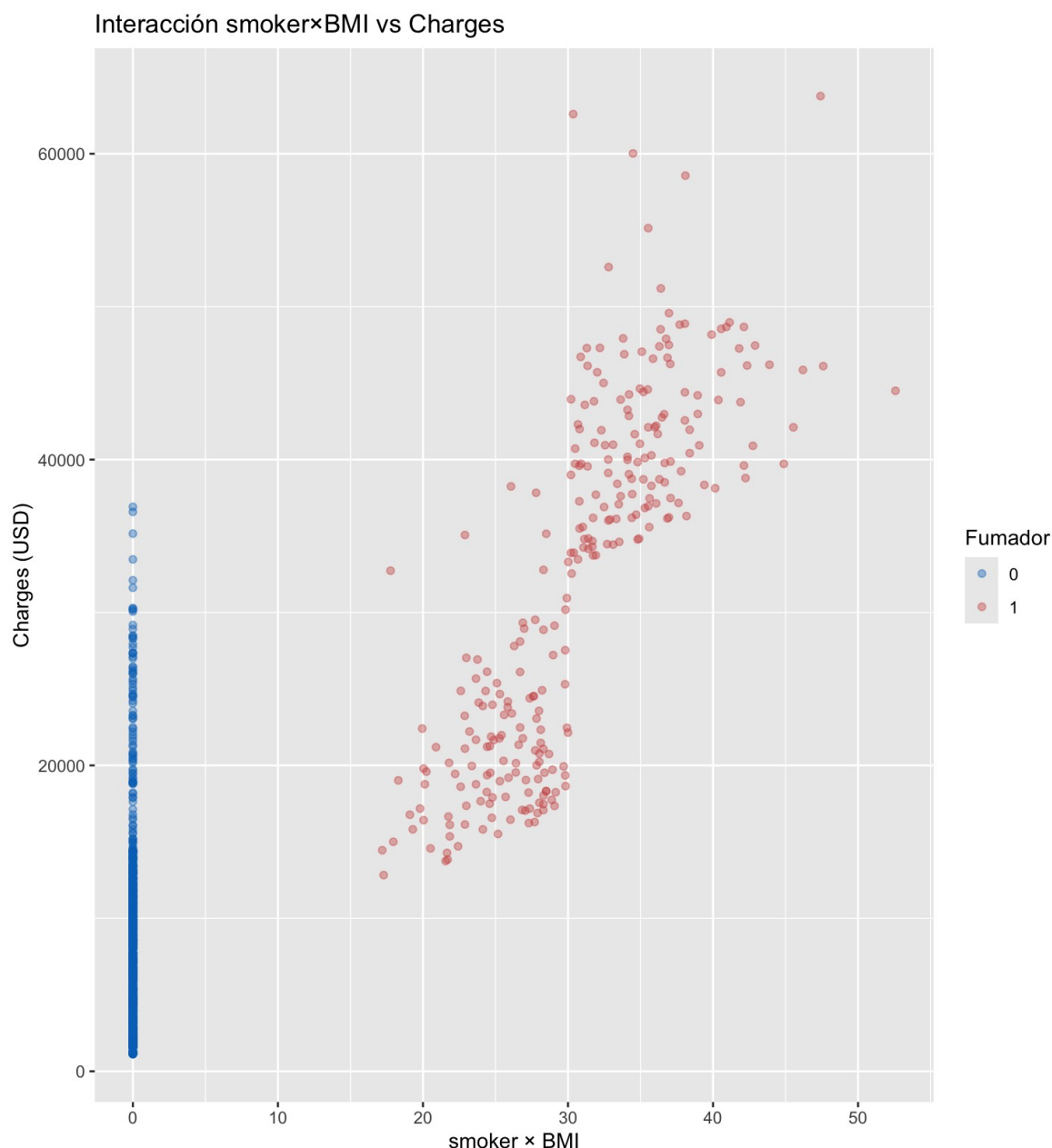


Figura 1. Interacción smoker x BMI vs charges

La figura expone con claridad el fenómeno multiplicativo que los modelos base no podían capturar. Los asegurados no fumadores (puntos azules) se concentran íntegramente en la columna vertical ubicada en $x = 0$, ya que al multiplicar $\text{smoker} = 0$ por cualquier valor de BMI el resultado siempre es cero. Este comportamiento revela por qué la variable BMI original resultaba poco informativa de forma aislada, su efecto sobre los costos es prácticamente nulo en ausencia del tabaquismo.

El escenario cambia radicalmente en el subgrupo de fumadores (puntos rojos). A medida que el valor de $\text{smoker} \times \text{BMI}$ aumenta hacia la derecha del eje horizontal, los cargos médicos escalan de forma progresiva y con una pendiente notablemente pronunciada, partiendo de aproximadamente \$13,000 USD y extendiéndose hasta superar los \$60,000 USD. Esta relación

lineal clara dentro del grupo fumador confirma que la nueva variable codifica información predictiva real que los algoritmos de la fase I no podían acceder con las variables originales separadas.

```
> ggplot(insurance_data, aes(x = age2, y = charges)) +  
+   geom_point(alpha = 0.3, color = "#868686FF") +  
+   geom_smooth(method = "lm", color = "red") +  
+   labs(title = "age2 vs Charges", x = "Edad2", y = "Charges  
(USD)")
```

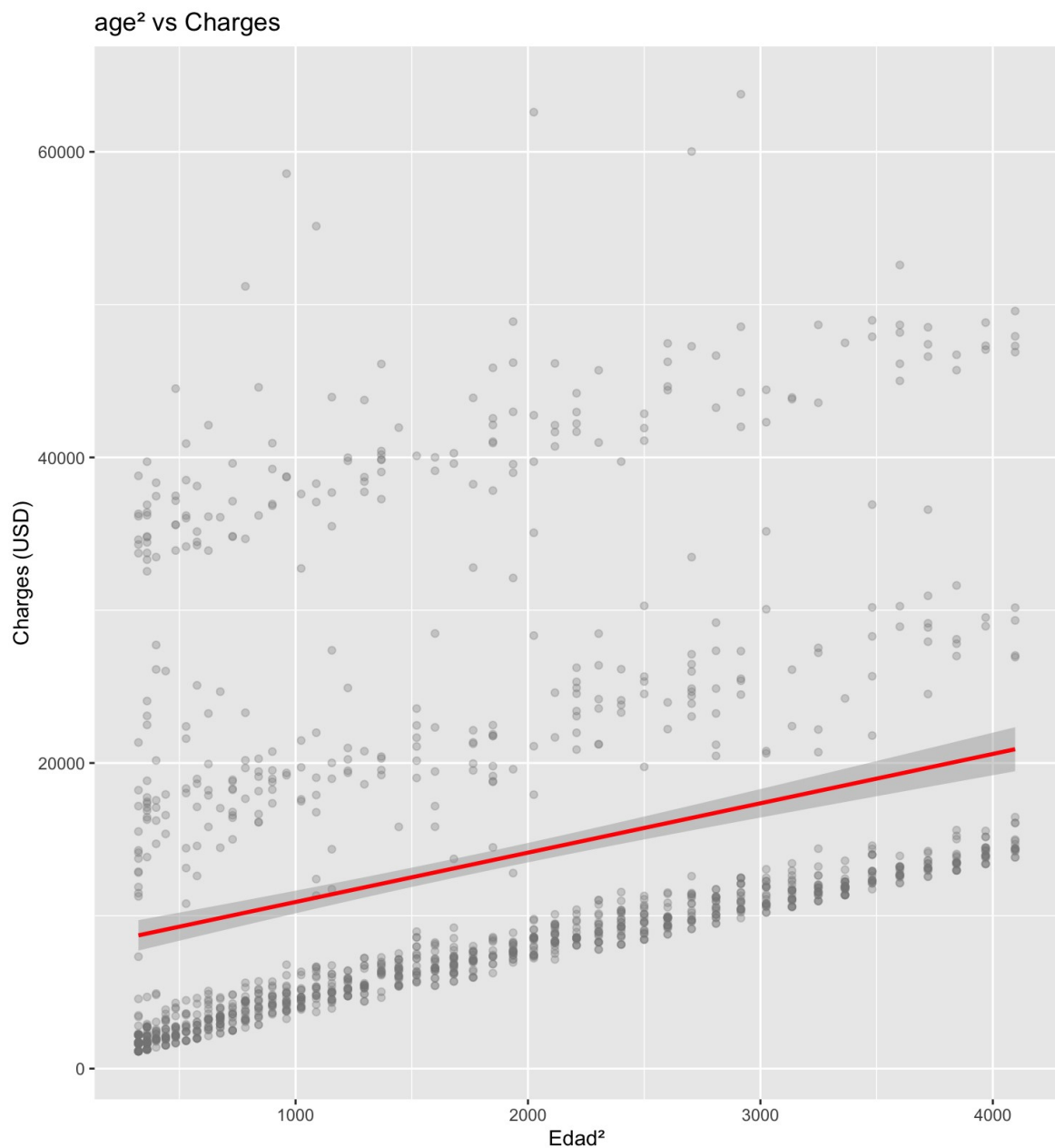


Figura 2. Diagrama de dispersión de la variable edad2 frente a los costos médicos, con línea de tendencia lineal.

El gráfico de dispersión de *age2* frente a *charges* revela una estructura de tres bandas horizontales paralelas, cuya existencia confirma que la transformación cuadrática capturó correctamente la dinámica del envejecimiento dentro de cada perfil de riesgo. La banda inferior, densa y concentrada, corresponde a los asegurados no fumadores, sus costos base son bajos pero exhiben una pendiente positiva constante, evidenciando que incluso en ausencia de factores de riesgo adicionales, el costo médico aumenta conforme avanza la edad al cuadrado. La banda intermedia pertenece a los sin obesidad clínica, quienes parten de un piso de costos más elevados pero siguen la misma tendencia ascendente. La banda superior, más dispersa, concentra a los fumadores con obesidad, el perfil de mayor riesgo, cuyos costos escalan con la mayor pendiente de las tres franjas.

La relevancia de esta visualización está en que las tres bandas muestran una inclinación positiva y paralela a lo largo del eje horizontal. Esto confirma que el envejecimiento opera como un multiplicador de costos universal que afecta de igual forma a todos los perfiles de riesgo, independientemente de la condición de tabaquismo u obesidad. La transformación cuadrática permitió al modelo capturar esta aceleración progresiva del gasto médico, reconociendo que el incremento de costos entre los 50 y los 64 años es cualitativamente más pronunciado que el registrado en etapas más tempranas de la vida adulta.

3.4. Impacto en la matriz de correlación

Con las nuevas variables integradas al conjunto de datos, se recalculó la matriz de correlación para evaluar si las variables derivadas aportan mayor señal predictiva que las originales.

```
> cor_matrix_v2 <- insurance_data %>%
+   select(age, age2, bmi, bmi_obese, children, sex, smoker,
+         smoker_bmi, smoker_age, smoker_obese,
+         region_northeast, region_northwest, region_southeast,
+         charges) %>%
+   cor() %>%
+   round(2)
> sort(cor_matrix_v2[, "charges"], decreasing = TRUE)
charges smoker_bmi smoker_obese smoker smoker_age
1.00      0.85      0.81      0.79      0.79 age
age2      bmi      bmi_obese children 0.30
0.30      0.20      0.20      0.07
region_southeast region_northeast region_northwest sex
0.07      0.01      -0.04      -0.06 >
corrplot(cor_matrix_v2,
+         method      = "color",
+         addCoef.col  = "black",
+         tl.col       = "black",
+         tl.srt       = 45,
+         title        = "Matriz de correlación -- variables
```

```
originales y derivadas",
+         mar         = c(0, 0, 1, 0))
```

Matriz de correlación — variables originales y derivadas

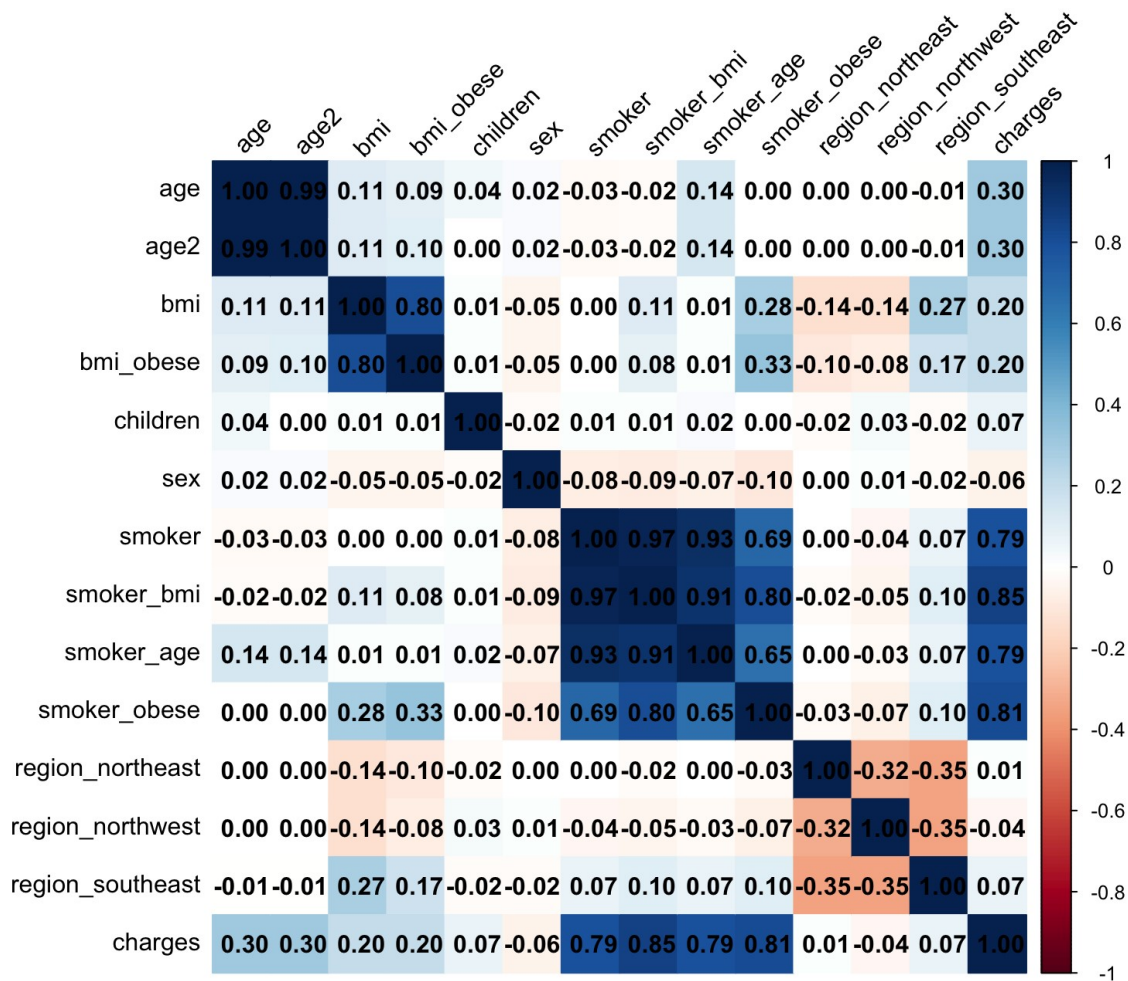


Figura 3. Mapa de calor de la matriz de correlación de Pearson incluyendo las variables originales y las cinco variables derivadas

El análisis del ranking de correlación con la variable objetivo revela el impacto directo de la ingeniería de características. En la fase I, la variable *smoker* era el predictor más fuerte con una correlación de 0.79. Tras la construcción de las variables derivadas, este valor fue superado por las tres nuevas variables de interacción; *smoker_bmi* alcanzó una correlación de 0.85, *smoker_obese* de 0.81 y *smoker_age* de 0.79. Este desplazamiento confirma que las interacciones construidas capturan información predictiva que las variables originales no

podían transmitir de forma aislada, y que la estrategia de ingeniería de características fue la intervención correcta para abordar las limitaciones de los modelos base.

3.5. Re-entrenamiento del modelo lineal con variables derivadas

Para cuantificar la ganancia predictiva atribuible exclusivamente a la ingeniería de características, se re-entrenó la *Regresión Lineal Múltiple* sobre el espacio de características ampliado. Previamente, se realizó la partición del conjunto de datos en un subconjunto de entrenamiento (70%, equivalente a 937 observaciones) y un subconjunto de prueba (30%, equivalente a 400 observaciones), fijando una semilla aleatoria para garantizar la reproducibilidad del experimento.

```
> set.seed(123)
> trainIndex <- createDataPartition(insurance_data$charges, p =
0.7, list = FALSE)
> train_set <- insurance_data[trainIndex, ]
> test_set <- insurance_data[-trainIndex, ]
```

El modelo fue entrenado incorporando las trece variables disponibles, las nueve originales más las cinco derivadas, utilizando la función *lm()* exclusivamente sobre el subconjunto de entrenamiento.

```
> modelo_lineal_v2 <- lm(charges ~ age + age2 + bmi + children +
sex + smoker +
+                               smoker_bmi + smoker_age + smoker_obese
+ bmi_obese +
+                               region_northeast + region_northwest +
region_southeast,
+                               data = train_set)
```

La decisión de incluir todas las variables simultáneamente responde a una estrategia deliberada, al exponer el modelo al espacio de características completo, el algoritmo puede estimar de forma honesta el peso real de cada predictor en presencia de todos los demás. Las variables que resulten redundantes o estadísticamente insignificantes serán identificadas por el propio modelo, sentando las bases para la selección automática de características que realizarán las regresiones penalizadas en la siguiente fase.


```

> summary(modelo_lineal_v2)

Call: lm(formula = charges ~ age + age2 + bmi + children + sex +
smoker
+      smoker_bmi + smoker_age + smoker_obese +
bmi_obese + region_northeast +
      region_northwest + region_southeast, data = train_set)

Residuals,
    Min       1Q   Median       3Q      Max
-2553   -1783   -1518   -834   23790

Coefficients,
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   1049.1314  1825.7669    0.575 0.565685
age          -59.0632    75.4045   -0.783 0.433661
age2           4.0599     0.9352    4.341 1.57e-05 ***
bmi           49.0612     0.728    0.466862    children      663.2195   130.2103
sex           5.093 4.26e-07 ***      511.6281   303.3565   1.687
smoker        0.092026 .      2107.0294  2792.8511   0.754
smoker_bmi    0.450779      434.6549   99.8091   4.355 1.48e-
05 *** smoker_age      -7.2713    26.9412  -0.270 0.787301
smoker_obese  16060.9834  1281.4956  12.533 < 2e-16 ***
bmi_obese     -186.3309    564.6326  -0.330 0.741472
region_northeast 1507.2994  430.3652   3.502 0.000483 ***
region_northwest 1163.0218  434.6074   2.676 0.007582 **
region_southeast  704.2879  426.6061   1.651 0.099097 .
---
Signif. codes,  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error, 4607 on 923 degrees of freedom
Multiple R-squared,  0.8502,    Adjusted R-squared,  0.8481
F-statistic,  403 on 13 and 923 DF,  p-value, < 2.2e-16

```

El resumen estadístico del modelo revela hallazgos relevantes en tres dimensiones. En primer lugar, los residuales presentan una mediana de -1,518, lo que indica que el modelo tiende a sobreestimar ligeramente los costos en la mayoría de los casos. El valor máximo de 23,790 señala que en el caso más extremo el modelo subestimó por \$23,790 USD, correspondiente a algún asegurado de perfil de muy alto costo. Esta asimetría en los residuales es esperada dado que la distribución de charges presenta una cola derecha pronunciada.

En segundo lugar, el análisis de significancia estadística de los coeficientes confirma la hipótesis de diseño. Las variables que obtienen la mayor certeza matemática son precisamente las derivadas mediante ingeniería de características, *age2* ($p=1.57e-05$), *smoker_bmi* ($p=1.48e-$

05) y *smoker_obese* ($p < 2e-16$). La variable *children* mantiene su relevancia estadística ($p = 4.26e-07$), mientras que variables como *age*, *bmi*, *smoker*, *smoker_age* y *bmi_obese* pierden significancia individual al quedar su información absorbida por las interacciones. Este fenómeno, lejos de ser problemático, valida la correcta construcción de las variables derivadas.

Finalmente, el Error Estándar Residual de 4,607 USD representa el error promedio del modelo sobre los datos de entrenamiento, expresado en la misma unidad monetaria que la variable objetivo. El R^2 Múltiple de 0.8502 indica que el modelo explica el 85% de la varianza en el conjunto de entrenamiento, mientras que el R^2 Ajustado de 0.8481, al penalizar por el número de variables incluidas, confirma que la mejora es genuina y no producto del incremento artificial de predictores.

3.6. Evaluación del impacto de la ingeniería de características

Con el modelo re-entrenado, se generaron predicciones sobre el conjunto de prueba y se calcularon las métricas de rendimiento para cuantificar la mejora

```
> pred_lineal_v2 <- predict(modelo_lineal_v2, newdata = test_set)
> mae_lineal_v2 <- mean(abs(pred_lineal_v2 - test_set$charges))
> rmse_lineal_v2 <- sqrt(mean((pred_lineal_v2 - test_set$charges)^2))
> r2_lineal_v2 <- cor(pred_lineal_v2, test_set$charges)^2
```

Lineal original	MAE= 4160.66	RMSE= 5655.73	$R^2=0.8085$
Lineal v2 con features	MAE= 2231.75	RMSE= 3870.81	$R^2=0.9099$

Los resultados evidencian que la Ingeniería de Características fue la intervención de mayor impacto en todo el proyecto. Manteniendo exactamente el mismo algoritmo de Regresión Lineal Múltiple, el simple enriquecimiento del espacio de características produjo una reducción del 46% en el Error Absoluto Medio, de \$4,160.66 a \$2,231.75 USD, una reducción del 32% en el RMSE, de \$5,655.73 a \$3,870.81 USD y un salto de 10 puntos porcentuales en el coeficiente de determinación, de 0.8085 a 0.9099. Notablemente, este modelo lineal enriquecido supera también al Árbol de la fase I ($R^2 = 0.8667$), que era hasta ese momento el mejor modelo del proyecto, sin necesidad de recurrir a algoritmos más complejos. Este hallazgo subraya que, antes de escalar la complejidad algorítmica, la inversión en la calidad y representatividad del espacio de características produce los retornos más significativos en rendimiento predictivo.

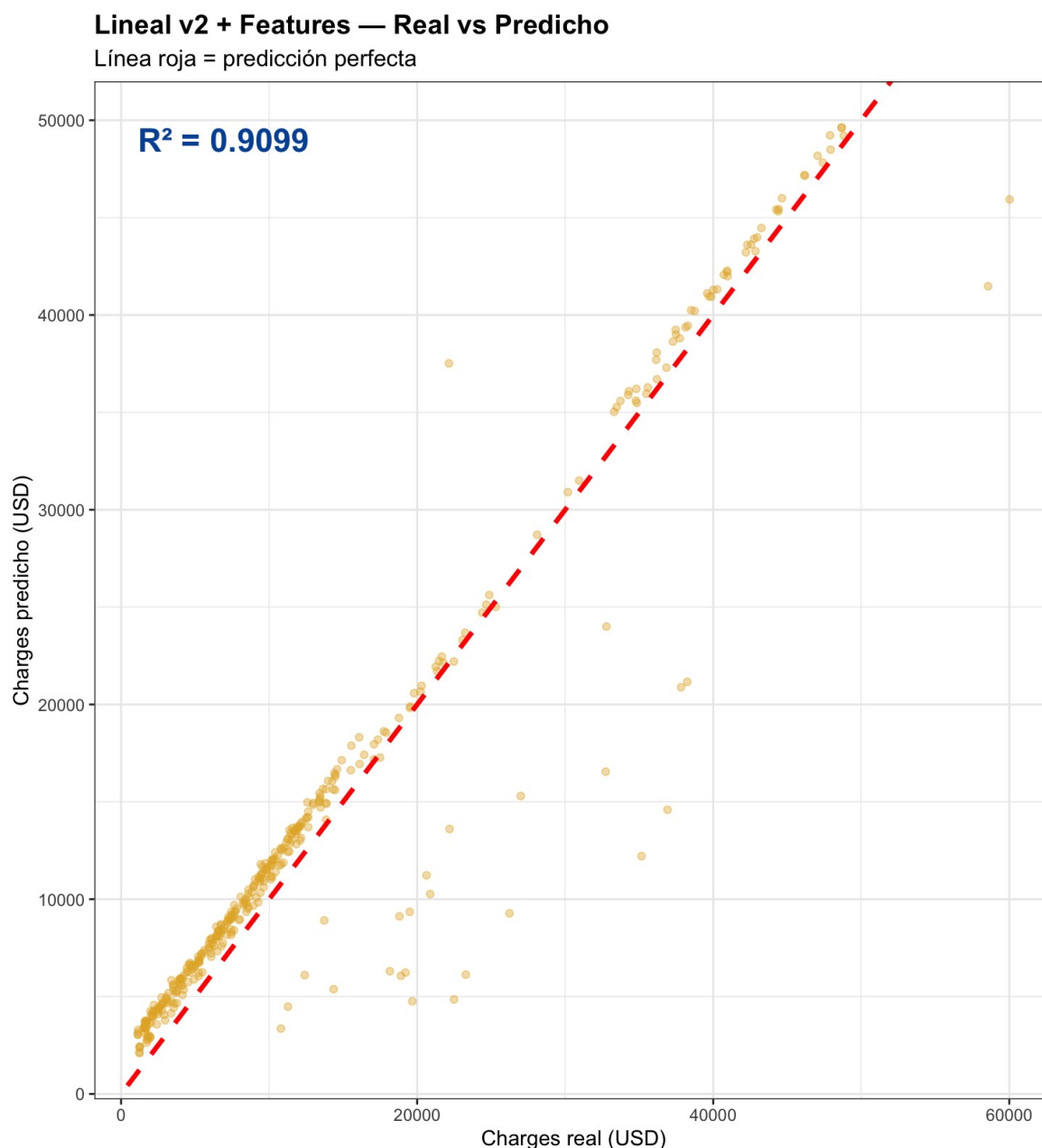


Figura 4. Modelo lineal versión con ingeniería de características

El gráfico de dispersión del modelo Lineal v2 con ingeniería de características constituye la visualización más reveladora del proyecto hasta este punto, ya que evidencia el salto cualitativo producido por la codificación explícita de las interacciones multiplicativas. La nube de puntos presenta una alineación notablemente más densa y consistente sobre la línea de predicción perfecta (trazada en rojo) en comparación con el modelo lineal original, reflejo directo del incremento en el coeficiente de determinación de 0.8085 a 0.9099.

El análisis visual permite identificar dos zonas diferenciadas. En el segmento de costos bajos y moderados (entre \$0 y \$20,000 USD), los puntos se agrupan con gran precisión sobre la diagonal, lo que indica que el modelo captura con exactitud el comportamiento de la mayoría de los asegurados. En el segmento de alto costo (superior a \$30,000 USD), corresponde al perfil de fumadores con obesidad clínica, la dispersión aumenta pero sigue siendo

considerablemente menor que en la versión original, confirmando que las variables *smoker_bmi* y *smoker_obese* lograron hacer accesibles para el algoritmo la información que antes permanecía implícita en los datos.

La presencia de algunos puntos alejados de la diagonal en la región, entre \$10,000 y \$30,000 USD) anticipa la limitación estructural que se abordará en las etapas siguientes, existe un subgrupo de asegurados cuyos costos reales superan las predicciones del modelo, probablemente asociados a condiciones médicas específicas que las variables disponibles en *insurance.csv* no logran capturar plenamente.

4. Regresiones penalizadas

4.1. Introducción y justificación

La ingeniería de características demostró que enriquecer el espacio de predictores produce mejoras sustanciales en el rendimiento. Sin embargo, este enriquecimiento conlleva un nuevo desafío, al pasar de 9 a 13 variables, algunas de ellas comparten información entre sí. Por ejemplo, *smoker* y *smoker_bmi* están matemáticamente relacionadas, al igual que *age* y *age2*. En presencia de esta correlación entre predictores, los coeficientes del modelo lineal ordinario se vuelven inestables y difíciles de interpretar.

Las regresiones penalizadas resuelven este problema añadiendo un término de penalización a la función de costo del modelo. Este término obliga al algoritmo a reducir los coeficientes de las variables menos informativas, mejorando la estabilidad del modelo y realizando selección automática de características. En esta fase se evalúan tres variantes, Ridge, Lasso y Lasso Adaptativo.

4.2. Preparación de los datos para glmnet

A diferencia de la función *lm()*, el paquete *glmnet* opera exclusivamente sobre matrices numéricas y no acepta *data.frames*. Esta exigencia técnica responde a que internamente el algoritmo resuelve el problema de optimización mediante álgebra matricial pura, lo que requiere un formato de datos estrictamente numérico y estructurado. La función *model.matrix()* realiza esta conversión, transformando la fórmula con todas las variables predictoras en una matriz donde cada fila representa un asegurado y cada columna una variable. El argumento *[-1]* elimina la columna del intercepto generada automáticamente por R, ya que *glmnet* calcula este término de forma interna.

```
> X_train <- model.matrix(charges ~ age + age2 + bmi + children +  
sex +  
+               smoker + smoker_bmi + smoker_age +  
+               smoker_obese + bmi_obese +  
+               region_northeast + region_northwest +  
+               region_southeast,  
+               data = train_set)[, -1] # -1 elimina el  
intercepto  
> y_train <- train_set$charges
```

```
> X_test <- model.matrix(charges ~ age + age2 + bmi + children +
sex +
                        smoker + smoker_bmi + smoker_age +
                        smoker_obese + bmi_obese +
                        region_northeast + region_northwest +
                        region_southeast,
                        data = test_set)[, -1]
> y_test <- test_set$charges
```

En este esquema, *y_train* y *y_test* contienen exclusivamente la variable objetivo (*charges*), extraída como vectores independientes de sus respectivos conjuntos. Esta separación es un requerimiento del paquete, que necesita recibir por separado la matriz de predictores *X* y el vector de respuesta *y*. La verificación de dimensiones confirma que la conversión preservó la integridad de los datos

```
> dim(X_train) # debe ser ~937 x 13 [1] 937 13
> dim(X_test)  # debe ser ~400 x 13
[1] 400 13
```

Los resultados son los esperados, 937 observaciones de entrenamiento y 400 de prueba, ambas con los 13 predictores definidos en la fórmula.

4.3. Regresión Ridge

La Regresión Ridge añade a la función de costo una penalización proporcional a la suma de los cuadrados de los coeficientes. Esta penalización reduce todos los coeficientes hacia cero de forma continua pero sin llevar ninguno exactamente a cero, lo que la convierte en una herramienta efectiva para estabilizar estimaciones en presencia de variables correlacionadas. Su principal ventaja es que conserva todas las variables en el modelo mientras distribuye su influencia de forma más equitativa.

Para encontrar el valor óptimo del parámetro de regularización, se utilizó validación cruzada de 10 pliegues mediante la función *cv.glmnet()*. El argumento *alpha=0* especifica la penalización Ridge, y *nfolds=10* indica que los 937 registros de entrenamiento se dividen en 10 subconjuntos iguales, en cada iteración el modelo se entrena sobre 9 subconjuntos y se evalúa sobre el restante, rotando hasta que todos han sido usados como validación

```
> set.seed(123)
> cv_ridge <- cv.glmnet(X_train, y_train,
+                       alpha = 0,      # 0 = Ridge
+                       nfolds = 10)
> cat("Lambda óptimo Ridge,", round(cv_ridge$lambda.min, 2), "\n")
Lambda óptimo Ridge, 974.37
```

El valor $\lambda = 974.37$ representa el punto de equilibrio óptimo entre ajuste y penalización. Lambda actúa como un regulador de complejidad, un valor de cero equivale a regresión lineal ordinaria sin restricciones, mientras que valores muy grandes reducen todos los coeficientes prácticamente a cero. El proceso de validación cruzada evaluó sistemáticamente decenas de

valores de lambda y determinó que 974.37 es el nivel de penalización que minimiza el error de predicción sobre datos no vistos.

```
> plot(cv_ridge)
> title("Ridge -- CV para selección de lambda", line = 3)
```

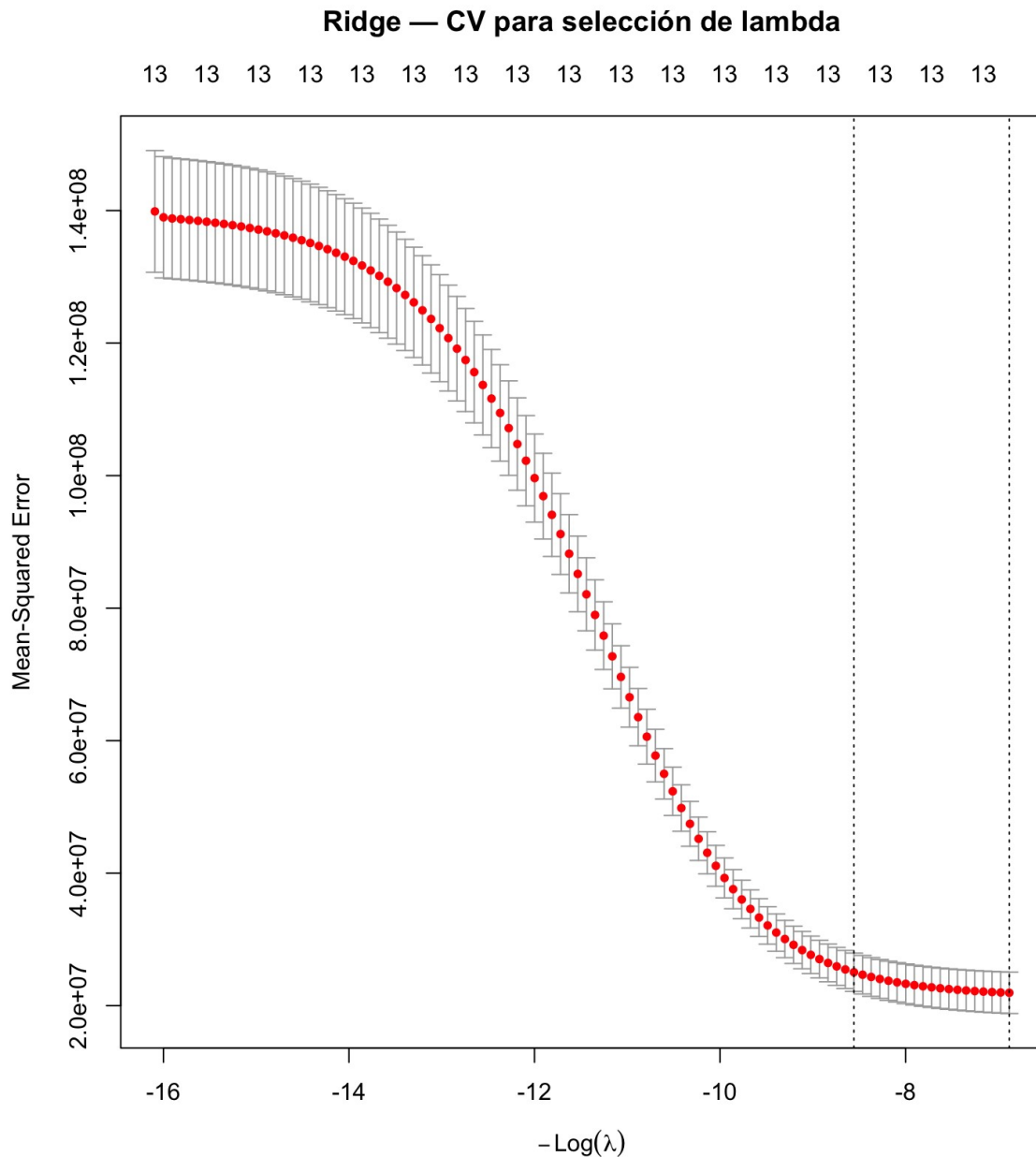


Figura 4. Curva de validación cruzada para la selección del lambda óptimo en la Regresión Ridge

La curva de validación cruzada muestra el error de predicción (MSE) en el eje vertical frente a los valores de $\log(\lambda)$ explorados en el eje horizontal. La trayectoria desciende hacia la derecha hasta alcanzar un mínimo, para luego elevarse nuevamente cuando la penalización se vuelve excesiva y el modelo pierde capacidad predictiva. La primera línea vertical punteada marca $\lambda_{\min}$, el valor que minimiza el MSE y que se utilizará para las predicciones. La

suavidad de la curva indica un modelo estable cuyo rendimiento no es sensible a pequeñas variaciones en el valor de lambda.

```
> pred_ridge <- as.vector(predict(cv_ridge,  
+                               s      = "lambda.min",  
+                               newx   = X_test))
```

La función *predict()* aplica el modelo Ridge entrenado al conjunto de prueba. El argumento *s = "lambda.min"* instruye al algoritmo a utilizar el lambda óptimo identificado por validación cruzada, y *newx = X_test* proporciona la matriz de predictores del conjunto de prueba sobre el cual se generan las estimaciones

```
MAE,  2264.89  
RMSE, 3897.38  
R2,  0.9099
```

Los resultados de Ridge son prácticamente idénticos a los del modelo Lineal v2 con features ($R^2=0.9099, 0.9099$ MAE=\$2,264 vs \$2,231, RMSE=\$3,897 vs \$3,870). Este comportamiento es esperado y no representa una limitación del algoritmo, sino una confirmación de que la ingeniería de características ya había capturado las relaciones más relevantes del problema. El valor de Ridge en este contexto no reside en superar al modelo lineal enriquecido, sino en estabilizar los coeficientes frente a la correlación entre las variables derivadas, estableciendo una base sólida para el siguiente paso, Lasso, que además de penalizar realiza selección automática de variables.

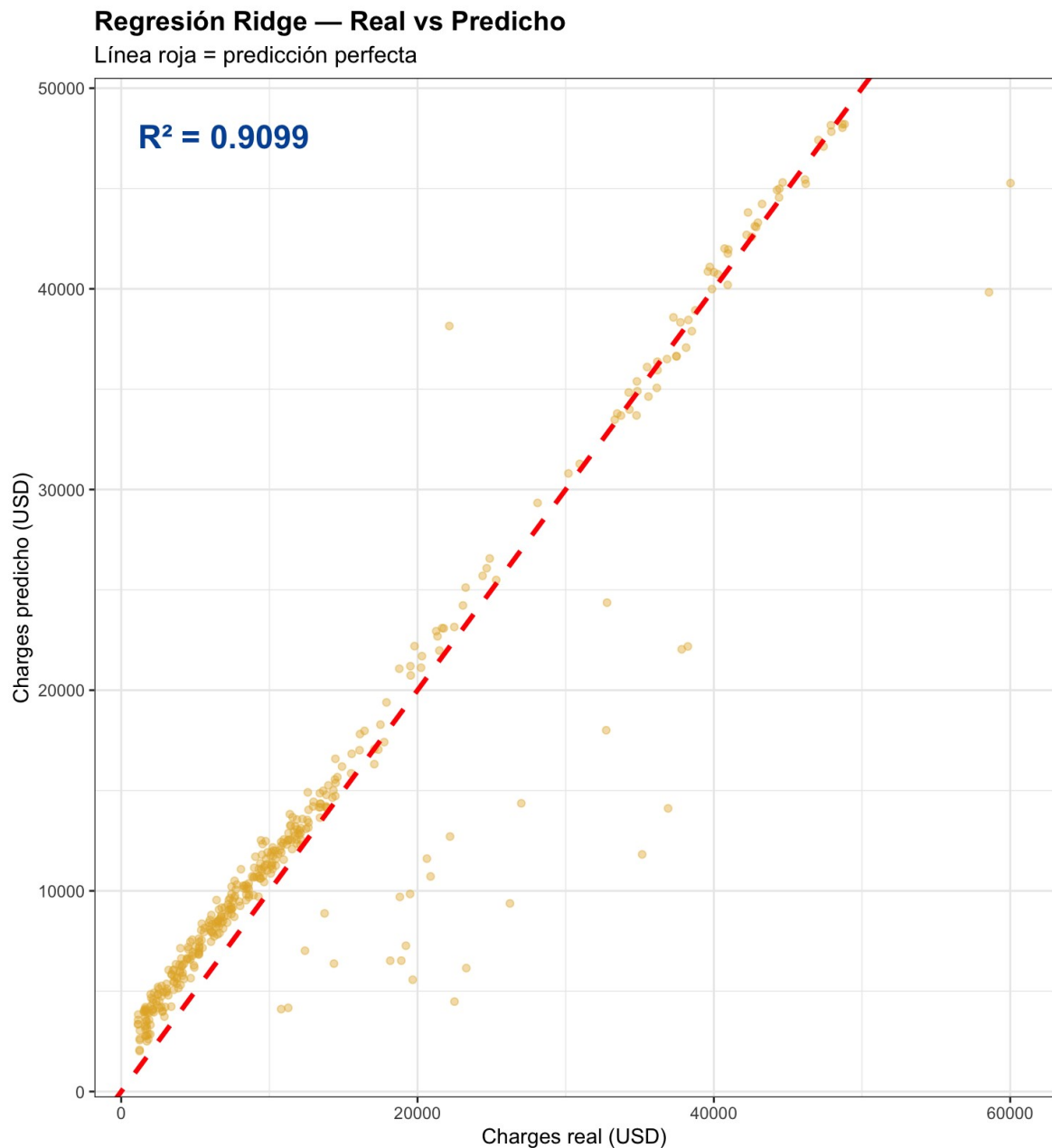


Figura 5. Modelo Ridge comparación valores reales contra predichos

El gráfico de dispersión de la Regresión Ridge reproduce con gran fidelidad el patrón visual del modelo Lineal v2 con ingeniería de características, resultado que es metodológicamente coherente con las métricas reportadas, ambos modelos alcanzan un R^2 idéntico de 0.9099. Esta similitud visual no representa una limitación del algoritmo Ridge, sino una confirmación de que su función principal en este contexto no era mejorar el poder predictivo bruto, sino estabilizar los coeficientes frente a la multicolinealidad introducida por las variables derivadas.

La distribución de los puntos mantiene la misma estructura tripartita observada en la versión sin penalización, alta concentración sobre la diagonal en el segmento de bajo costo, dispersión

moderada en el rango intermedio, y puntos atípicos en la región de alto costo. Esta invarianza visual confirma que Ridge redistribuye los pesos entre variables correlacionadas sin alterar el patrón global de predicción, comportamiento matemáticamente esperado dado que su penalización ℓ_2 contrae los coeficientes de forma continua pero nunca los elimina. La superposición práctica entre ambos gráficos establece visualmente la base de partida sobre la que Lasso intentará mejorar mediante selección activa de variables.

4.4. Regresión Lasso

La Regresión Lasso (Least Absolute Shrinkage and Selection Operator) comparte con Ridge el objetivo de penalizar la complejidad del modelo, pero difiere en el tipo de penalización aplicada. Mientras Ridge utiliza la suma de los cuadrados de los coeficientes (penalización ℓ_2), Lasso penaliza la suma de sus valores absolutos (penalización ℓ_1). Esta diferencia matemática, aparentemente sutil, tiene una consecuencia práctica de gran relevancia, la penalización ℓ_1 puede llevar coeficientes exactamente a cero, eliminando variables del modelo de forma automática. Lasso realiza simultáneamente regularización y selección de características, produciendo modelos más parsimoniosos e interpretables.

El modelo se ajustó con los mismos criterios de validación cruzada que Ridge, modificando únicamente el argumento *alpha=1* para especificar la penalización Lasso

```
> set.seed(123)
> cv_lasso <- cv.glmnet(X_train, y_train,
+                       alpha = 1,      # 1 = Lasso
+                       nfolds = 10)
>
> cat("Lambda óptimo Lasso,", round(cv_lasso$lambda.min, 2), "\n")
Lambda óptimo Lasso, 40.26
```

El lambda óptimo de 40.26 es notablemente inferior al de Ridge (974.37). Esta diferencia no implica que Lasso penalice menos, sino que su mecanismo ℓ_1 es matemáticamente más agresivo por naturaleza, un lambda pequeño en Lasso produce un nivel de contracción equivalente a un lambda mucho mayor en Ridge. La validación cruzada de 10 pliegues determinó que este es el valor donde el modelo logra el mejor balance entre eliminación de variables irrelevantes y retención de señal predictiva.

```
> plot(cv_lasso)
> title("Lasso -- CV para selección de lambda", line = 3)
```

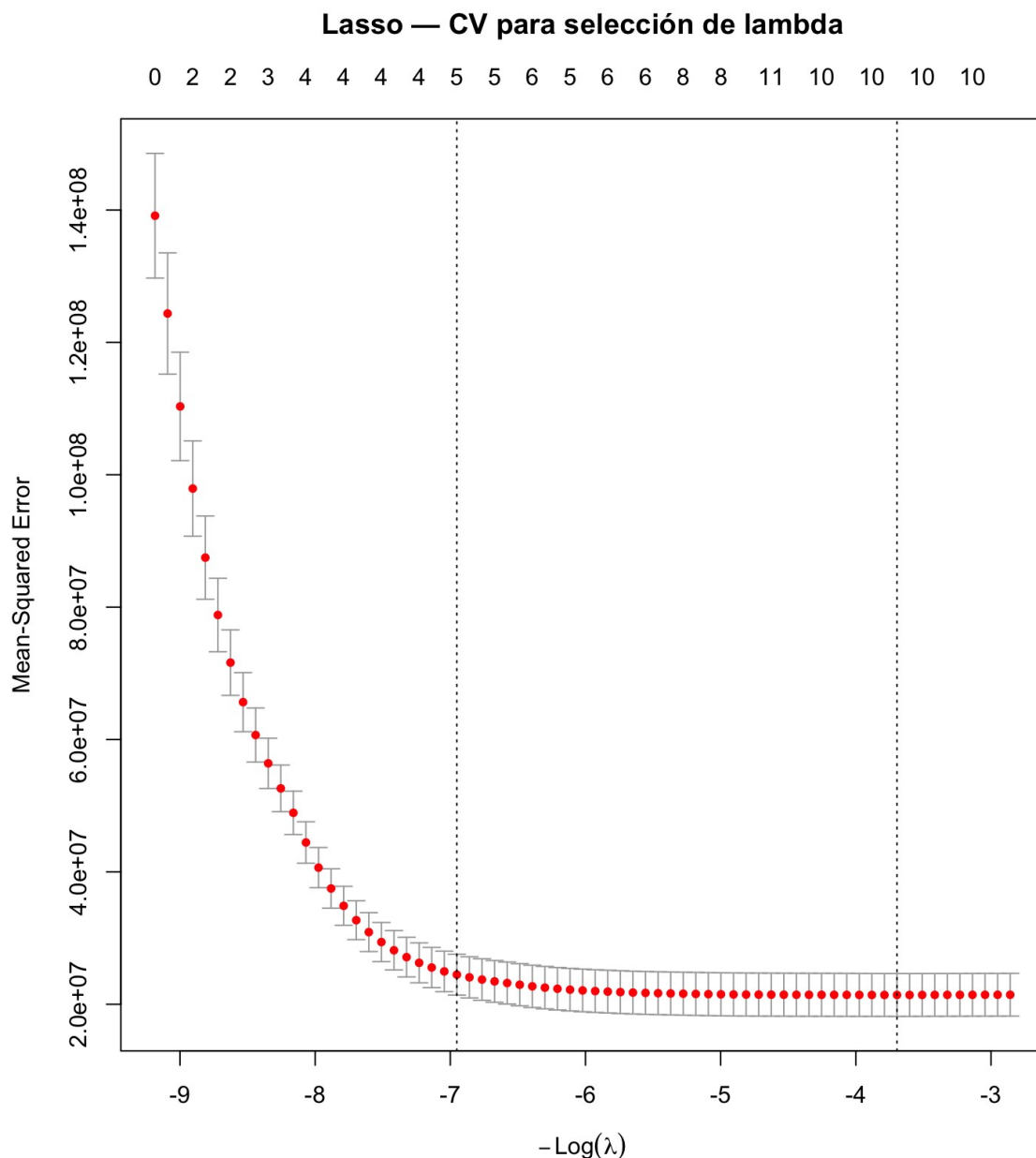


Figura 6.. Curva de validación cruzada para la selección del lambda óptimo en la Regresión Lasso.

La curva de validación cruzada de Lasso incorpora un elemento informativo adicional respecto a la de Ridge, los números en el eje horizontal superior indican cuántas variables permanecen activas (coeficiente distinto de cero) para cada valor de lambda evaluado. Al desplazarse hacia la derecha, la penalización aumenta y el número de variables activas decrece progresivamente hasta llegar a cero. La línea vertical en *lambda.min* identifica el punto de mínimo error, donde el modelo retiene el mayor número de variables aún informativas antes de que la penalización comience a degradar el rendimiento predictivo.

```
> pred_lasso <- as.vector(predict(cv_lasso,
+                               s      = "lambda.min",
+                               newx   = X_test))
MAE, 2210.26
```

```
RMSE, 3851.45
R², 0.9108
```

Lasso registra una mejora consistente en las tres métricas respecto a Ridge y al modelo Lineal v2, el MAE desciende a \$2,210.26, el RMSE a \$3,851.45 y el R^2 asciende a 0.9108. Aunque la ganancia es marginal en términos absolutos, es significativa metodológicamente porque confirma que la selección automática de variables añade valor real, eliminando el ruido estadístico de los predictores redundantes.

Para comprender el origen de esta mejora, se analizaron los coeficientes asignados por ambos modelos penalizados

```
> coefs <- cbind(
+   Ridge = round(as.vector(coef(cv_ridge, s = "lambda.min")), 4),
+   Lasso = round(as.vector(coef(cv_lasso, s = "lambda.min")), 4) +
+ )
> rownames(coefs) <- rownames(coef(cv_ridge, s = "lambda.min")) >
print(coefs)
```

	Ridge	Lasso
(Intercept)	-1942.4180	988.7237
age		
101.8451	0.0000	age2
1.7626	3.2842	bmi
69.9831	14.0849	children
517.7499	597.1878	sex
432.7640	431.6210	smoker
5059.4586	621.8420	smoker_bmi
255.1442	479.4634	smoker_age
52.2057	0.0000	smoker_obese
15693.8935	15525.2920	bmi_obese
4.6591	0.0000	region_northeast
1209.3283	1213.8648	region_northwest
909.3650	867.1289	region_southeast
490.7947	444.0589	

La tabla comparativa expone la diferencia fundamental entre ambos enfoques. Ridge conserva todas las variables activas pero reduce sus coeficientes hacia valores menores, distribuyendo la penalización de forma continua. Lasso, en cambio, eliminó completamente tres variables asignándoles un coeficiente de exactamente 0.0000, *age* (cuya información quedó absorbida por *age2*), *smoker_age* (redundante frente a la mayor especificidad de *smoker_obese*) y *bmi_obese* (absorbida por la interacción *smoker_obese*). Este resultado coincide con exactitud con las variables que el *summary()* del modelo lineal v2 marcó como estadísticamente no significativas, validando de forma independiente ambos análisis.

El coeficiente de *smoker* experimentó una contracción drástica, de 5,059 en Ridge a 621 en Lasso, porque el algoritmo reconoció que prácticamente toda la información predictiva del tabaquismo ya está capturada por las variables de interacción *smoker_bmi* y *smoker_obese*. Por su parte, *smoker_obese* mantiene el coeficiente más alto en ambos modelos (~15,500 USD), reafirmando que la combinación de tabaquismo y obesidad clínica es el predictor de mayor peso financiero del conjunto de datos.

Lineal original	MAE= 4160.66	RMSE= 5655.73	R ² =0.8085
Lineal v2 + features	MAE= 2231.75	RMSE= 3870.81	R ² =0.9099
Ridge	MAE= 2264.89	RMSE= 3897.38	R ² =0.9099
Lasso	MAE= 2210.26	RMSE= 3851.45	R ² =0.9108

La tabla acumulada confirma que Lasso se posiciona como el mejor modelo penalizado hasta el momento, superando en las tres métricas tanto a Ridge como al modelo lineal enriquecido. Su capacidad de selección automática de variables, al eliminar predictores redundantes sin sacrificar poder explicativo, lo establece como la referencia a superar en la siguiente etapa, el Lasso Adaptativo, que refina aún más este mecanismo de selección.

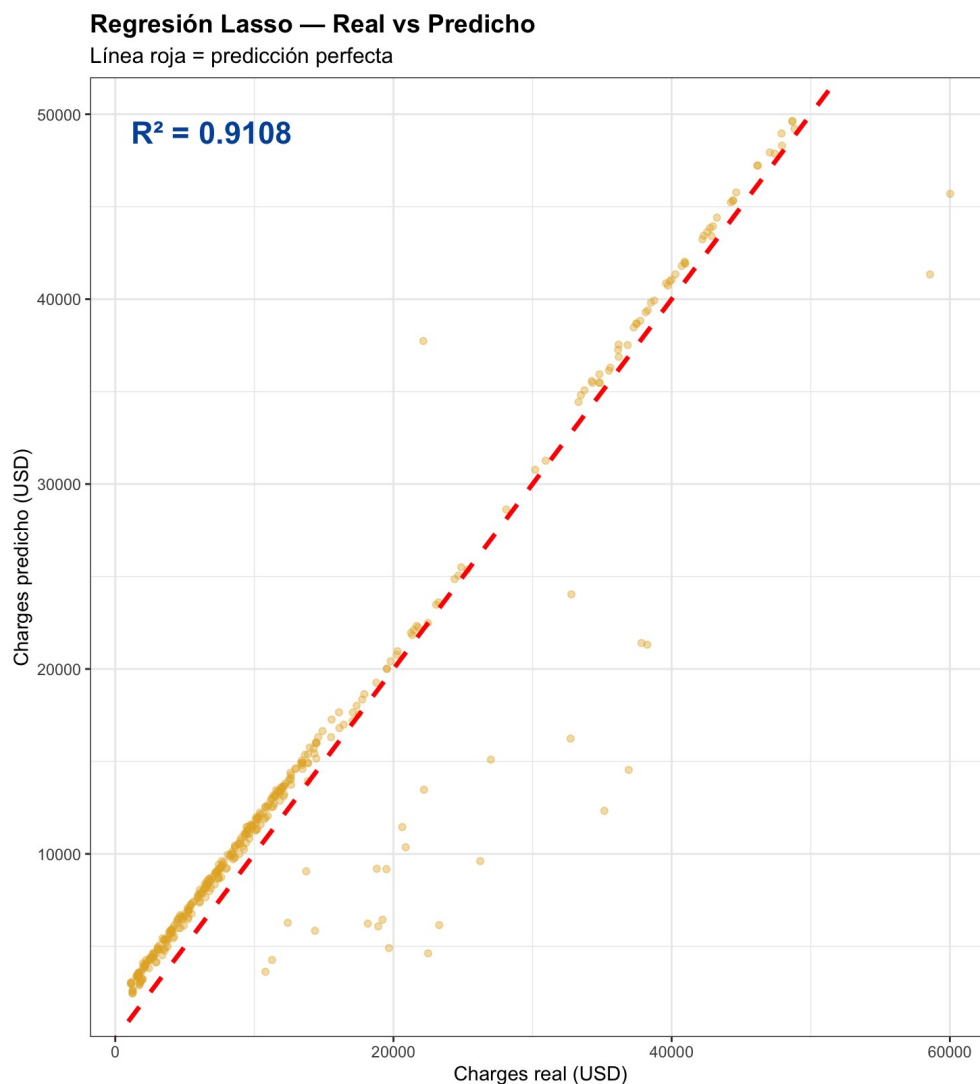


Figura 7. Modelo Lasso, gráfica de datos reales contra los predichos

El gráfico de dispersión del modelo Lasso refleja la mejora marginal pero consistente que este algoritmo introduce respecto a Ridge, con un R^2 de 0.9108 frente a 0.9099. Aunque la diferencia es difícilmente perceptible a simple vista, un análisis cuidadoso de la distribución de puntos revela una ligera reducción en la dispersión de la región intermedia, atribuible a la eliminación de tres predictores redundantes que Ridge mantenía activos con coeficientes pequeños, *age*, *smoker_age* y *bmi_obese*.

La concentración de puntos sobre la línea de predicción perfecta en el segmento de bajo y medio costo es prácticamente idéntica a la de Ridge, lo que confirma que la selección automática de variables de Lasso no perjudica la precisión en los perfiles de riesgo más frecuentes. El patrón de dispersión en el extremo superior del eje (costos superiores a \$40,000 USD) se mantiene similar, evidenciando que la limitación en ese segmento no es atribuible a variables redundantes sino a la variabilidad intrínseca de los perfiles de alto riesgo, que ningún modelo lineal puede eliminar completamente con las variables disponibles.

4.5. Lasso Adaptativo

El Lasso Adaptativo, introducido por Zou (2006), representa una mejora teórica y práctica sobre el Lasso estándar. Su aportación principal consiste en reemplazar la penalización uniforme aplicada por Lasso a todas las variables por una penalización diferenciada e inteligente, las variables identificadas como importantes reciben poca penalización y se conservan en el modelo, mientras que las variables irrelevantes son penalizadas con mayor intensidad y eliminadas con mayor decisión. Esta propiedad le permite al Lasso Adaptativo alcanzar las denominadas propiedades Oracle, es decir, comportarse asintóticamente como si conociera de antemano cuáles variables son verdaderamente relevantes.

La implementación del Lasso Adaptativo requiere un procedimiento de dos etapas. En la primera, se ajusta un modelo Ridge sobre los datos estandarizados para obtener estimaciones iniciales de los coeficientes. En la segunda, estos coeficientes se utilizan para calcular los pesos adaptativos que guiarán la penalización diferenciada del modelo final.

4.5.1. Estandarización de los datos

El Lasso Adaptativo exige que las variables estén en una escala comparable antes de calcular los pesos, ya que los coeficientes Ridge se utilizarán directamente para determinar la importancia relativa de cada predictor. Si las variables operan en escalas muy dispares (por ejemplo, *age2* con valores entre 324 y 4,096 frente a *bmi_obese* con valores de 0 o 1), el modelo podría malinterpretar la magnitud de los coeficientes asignando pesos incorrectos

```
> medias <- colMeans(X_train)
> desv   <- apply(X_train, 2, sd)
```

colMeans() calcula la media de cada columna de la matriz de entrenamiento y *apply(X_train, 2, sd)* calcula su desviación estándar. Ambos parámetros se almacenan en variables

independientes porque deben aplicarse de forma idéntica tanto al conjunto de entrenamiento como al de prueba. Estandarizar el conjunto de prueba con sus propios parámetros introduciría información futura en el modelo, un error metodológico conocido como fuga de datos (*data leakage*) que produciría estimaciones de rendimiento artificialmente optimistas.

```
> X_train_s <- scale(X_train, center = medias, scale = desv)
> X_test_s <- scale(X_test, center = medias, scale = desv)
> dim(X_train_s) # 937 x 13 [1]
937 13
> dim(X_test_s) # 400 x 13
[1] 400 13
```

La función *scale()* transforma cada variable restando su media (centrado) y dividiéndola entre su desviación estándar (escalado), llevando todas las columnas a una distribución de media cero y desviación estándar uno. La verificación de dimensiones confirma que la estandarización preservó la integridad del dato, 937 observaciones de entrenamiento y 400 de prueba, ambas con los 13 predictores.

4.5.2. Ridge sobre datos estandarizados

Con los datos en escala comparable, se ajusta un modelo Ridge mediante validación cruzada de 10 pliegues. El argumento *alpha=0* especifica la penalización Ridge, cuya función es proporcionar estimaciones iniciales de los coeficientes estables y continuas para todas las variables, condición necesaria para que los pesos adaptativos resulten informativos

```
> set.seed(123)
> cv_ridge_s <- cv.glmnet(X_train_s, y_train,
+                          alpha = 0,
+                          nfolds = 10)
> cat("Lambda Ridge estandarizado,", round(cv_ridge_s$lambda.min,
2), "\n")
Lambda Ridge estandarizado, 974.37
```

El lambda óptimo de 974.37, idéntico al del modelo Ridge original, confirma que el punto de equilibrio entre ajuste y regularización es el mismo independientemente de la escala de los datos, ya que la validación cruzada lo determina de forma adaptativa en cualquier espacio métrico.

4.5.3. Cálculo de los pesos adaptativos

A partir de los coeficientes del Ridge estandarizado se calculan los pesos que gobernarán la penalización diferenciada del modelo final

```
> coef_ridge_s <- as.matrix(coef(cv_ridge_s, s = "lambda.min")) >
g <- 1
> w_adapt_s <- 1 / (abs(coef_ridge_s[-1, ]) ^ g)
```

La fórmula $1 / (|\text{coeficiente}|^g)$ establece una relación inversa entre la importancia de una variable y su penalización, una variable con un coeficiente Ridge grande (predictor fuerte)

recibe un peso pequeño y será protegida por el modelo, mientras que una variable con coeficiente Ridge pequeño (predictor débil) recibe un peso grande y será penalizada con mayor intensidad. El parámetro $\gamma=1$ establece una relación de proporcionalidad directa, siendo el valor más comúnmente recomendado en la literatura

```
> round(sort(w_adapt_s, decreasing = TRUE), 4)
bmi_obese sex
region_southeast region_northwest      bmi    0.4293    0.0046
0.0046      0.0026      0.0024 region_northeast
children smoker_age age      age2    0.0019      0.0016
0.0012    0.0007    0.0005 smoker smoker_bmi smoker_obese
0.0005    0.0003      0.0002
```

El ranking de pesos revela un hallazgo aparentemente paradójico, *bmi_obese*, variable eliminada por Lasso, obtiene el peso más alto (0.4293), mientras que *smoker_obese*, el predictor más importante del proyecto, recibe el peso más bajo (0.0002). La explicación reside en el comportamiento de Ridge sobre datos estandarizados. *bmi_obese* es una variable binaria (0 o 1) cuya información ya está capturada en gran medida por *smoker_obese*; en consecuencia, Ridge le asigna un coeficiente pequeño en escala estandarizada, lo que se traduce en un peso alto y una penalización intensa en el Adaptive Lasso. Inversamente, *smoker_obese* recibe el coeficiente estandarizado más grande, lo que le otorga el peso más bajo y garantiza su permanencia en el modelo final. Las variables de región geográfica y *sex* presentan pesos intermedios, confirmando su contribución marginal pero no nula al poder predictivo.

4.5.4. Ajuste del Lasso Adaptativo

Con los pesos calculados, se ajusta el modelo Adaptive Lasso incorporando la penalización diferenciada mediante el argumento *penalty.factor*

```
> set.seed(123)
> cv_lasso_s <- cv.glmnet(X_train_s, y_train,
+                        alpha          = 1,
+                        penalty.factor = w_adapt_s,
+                        nfolds        = 10)
> cat("\nLambda óptimo ALasso corregido,",
round(cv_lasso_s$lambda.min, 2), "\n")
Lambda óptimo ALasso corregido, 814.22
```

El argumento *alpha=1* especifica la penalización ℓ_1 propia de Lasso, y *penalty.factor = w_adapt_s* sustituye la penalización uniforme de Lasso estándar por los pesos calculados en la etapa anterior. El lambda óptimo de 814.22 es mayor que el de Lasso estándar (40.26) pero menor que Ridge (974.37). Este valor más alto respecto a Lasso se explica porque los pesos adaptativos ya protegen a las variables importantes de la penalización excesiva, permitiendo al algoritmo aplicar un lambda global más intenso sin deteriorar el rendimiento en los predictores clave.

Las predicciones se generan aplicando el modelo al conjunto de prueba estandarizado. Es fundamental utilizar *X_test_s* (y no *X_test*) porque el modelo fue entrenado en el espacio

estandarizado; aplicarlo sobre datos en escala original produciría predicciones incorrectas al no corresponder las magnitudes esperadas

```
> pred_lasso_s <- as.vector(predict(cv_lasso_s,
+                               s = "lambda.min",
+                               newx = X_test_s))
MAE, 2205.63
RMSE, 3843.18
R², 0.9111
```

El Lasso Adaptativo corregido alcanza el mejor rendimiento de todos los modelos penalizados evaluados hasta el momento, con un $R^2=0.9111$, MAE=\$2,205.63 y RMSE=\$3,843.18. La mejora respecto a Lasso es consistente en las tres métricas, confirmando que la penalización diferenciada e inteligente añade valor real al proceso de selección de características.

```
> coefs_final <- cbind(
+   Ridge = round(as.vector(coef(cv_ridge, s = "lambda.min")),
+   4),
+   Lasso = round(as.vector(coef(cv_lasso, s = "lambda.min")),
+   4),
+   ALasso = round(as.vector(coef(cv_lasso_s, s = "lambda.min")),
+   4)
+ )
> rownames(coefs_final) <- rownames(coef(cv_ridge, s =
+ "lambda.min"))
> print(coefs_final)
```

	Ridge	Lasso	ALasso
(Intercept)	-1942.4180	988.7237	13136.8906
age	101.8451	0.0000	0.0000
age2	1.7626	3.2842	3748.7895
bmi	69.9831	14.0849	55.6104
children	517.7499	597.1878	725.3368
sex	432.7640	431.6210	149.9741
smoker	5059.4586	621.8420	131.2224
smoker_bmi	255.1442	479.4634	6132.7330
smoker_age	52.2057	0.0000	0.0000
smoker_obese	15693.8935	15525.2920	4597.6750
bmi_obese	4.6591	0.0000	0.0000
region_northeast	1209.3283	1213.8648	450.6581
region_northwest	909.3650	867.1289	285.1060
region_southeast	490.7947	444.0589	74.8642

La tabla comparativa de los tres modelos penalizados expone con claridad la evolución en la selección de variables. El Lasso Adaptativo elimina exactamente las mismas tres variables que Lasso estándar, *age*, *smoker_age* y *bmi_obese*, validando de forma independiente que estos

predictores son genuinamente redundantes. Es importante notar que los coeficientes del Adaptive Lasso no son directamente comparables en magnitud con los de Ridge y Lasso, ya que se expresan en unidades estandarizadas (efecto por desviación estándar) mientras los otros dos operan en escala original (USD por unidad). Lo que sí es comparable y significativo es el patrón de ceros y no-ceros, los tres modelos coinciden en identificar a *smoker_obese*, *smoker_bmi* y *age2* como los predictores de mayor peso relativo, reafirmando consistentemente los hallazgos del análisis exploratorio de la fase I.

Lineal original	MAE= 4160.66	RMSE= 5655.73	$R^2=0.8085$
Lineal v2 + features	MAE= 2231.75	RMSE= 3870.81	$R^2=0.9099$
Ridge	MAE= 2264.89	RMSE= 3897.38	$R^2=0.9099$
Lasso	MAE= 2210.26	RMSE= 3851.45	$R^2=0.9108$
ALasso corregido	MAE= 2205.63	RMSE= 3843.18	$R^2=0.9111$

La tabla acumulada confirma que el Lasso Adaptativo corregido es el mejor modelo de la fase de regresiones penalizadas y el mejor del proyecto hasta este punto. La progresión de mejoras ilustra cómo cada intervención metodológica aportó valor incremental, la ingeniería de características produjo la ganancia más significativa al pasar de $R^2 = 0.8085$ a 0.9099 , mientras que las regresiones penalizadas refinaron este resultado de forma gradual hasta alcanzar $R^2 = 0.9111$. Con el techo de los modelos lineales establecido, la siguiente fase introducirá los métodos de ensamble, cuya capacidad para capturar patrones no lineales complejos abre la posibilidad de superar esta barrera.

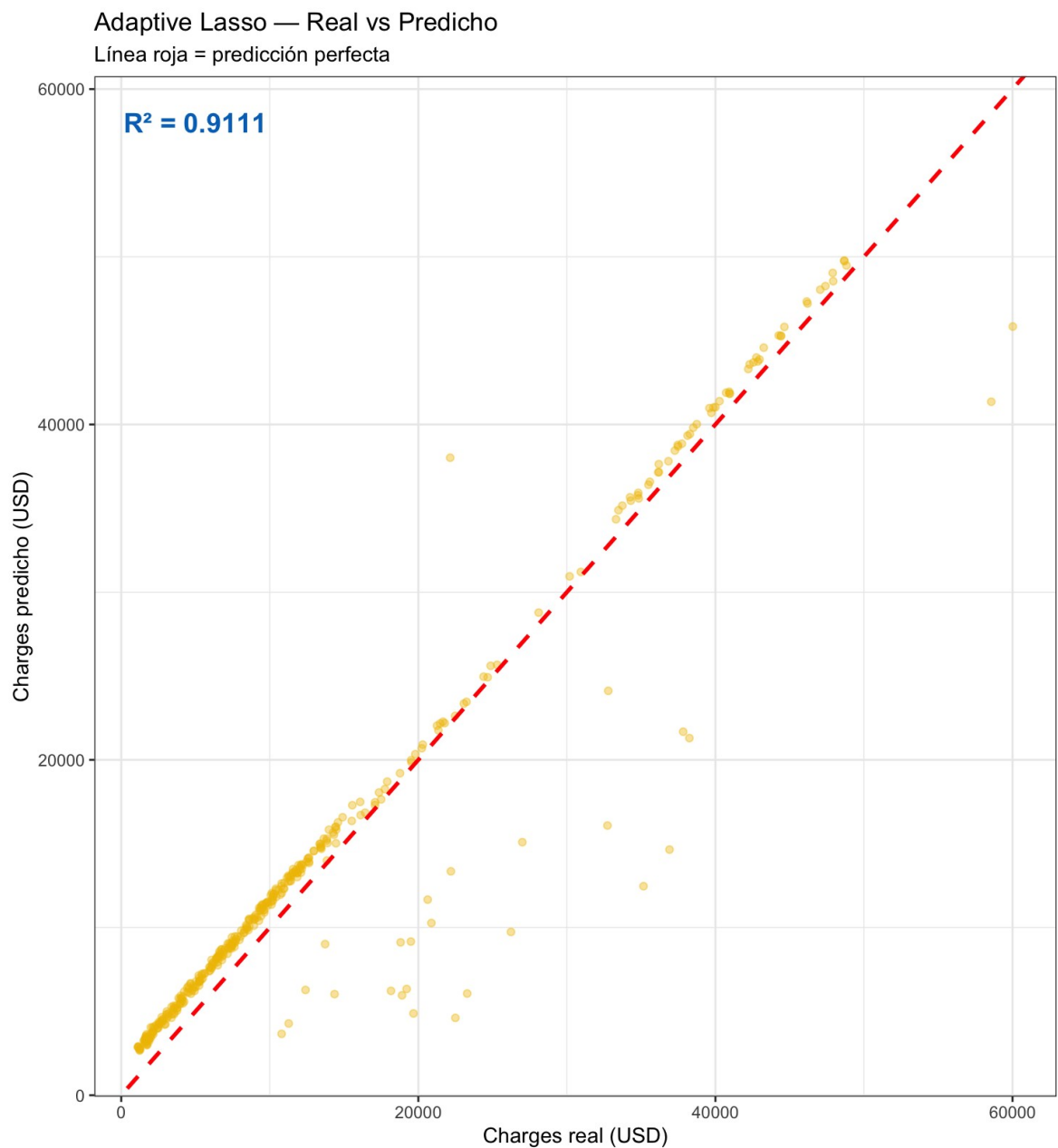


Figura 8. Gráfico de dispersión de los cargos médicos reales frente a los predichos por el modelo Lasso Adaptativo sobre el conjunto de prueba.

5. Métodos de ensamble

5.1. Random Forest base

Con el techo de los modelos lineales penalizados establecido en $R^2 = 0.9111$, la siguiente etapa introduce los métodos de ensamble. A diferencia de los modelos R entrenados hasta este punto,

que construyen una única función de predicción, Random Forest entrena cientos de árboles de decisión de forma independiente y promedia sus predicciones. Este mecanismo de agregación reduce la varianza del modelo sin incrementar el sesgo, permitiendo capturar patrones no lineales complejos que ningún modelo individual podría modelar de forma robusta.

El modelo base fue entrenado con la función `randomForest()` utilizando dos hiperparámetros principales

```
> set.seed(123)
> rf_base <- randomForest(charges ~ .,
+                           data = train_set, +
+                           ntree = 500,
+                           importance = TRUE)
```

El valor `ntree = 500` define el número de árboles que componen el bosque. Este parámetro no se optimiza mediante validación cruzada porque su efecto sobre el rendimiento predictivo es monótono, a mayor número de árboles, el error de predicción converge hacia un valor estable sin riesgo de sobreajuste. El valor de 500 es el estándar ampliamente aceptado en la literatura, suficiente para garantizar la estabilidad del promedio sin incurrir en un costo computacional innecesario. El argumento `importance = TRUE` instruye al algoritmo a registrar la contribución de cada variable al poder predictivo del modelo, información que se analizará en la siguiente sección.

El resumen del modelo entrenado reporta los siguientes resultados sobre el conjunto de entrenamiento

```
> print(rf_base)

Call, randomForest(formula = charges ~ ., data = train_set, ntree
=
500,      importance = TRUE)
      Type of random forest, regression
      Number of trees, 500
No. of variables tried at each split, 4

      Mean of squared residuals, 24160356
      % Var explained, 82.69
```

El campo "*No. of variables tried at each split, 4*" merece una explicación detallada. En cada nodo de cada árbol, Random Forest no evalúa todas las variables disponibles para decidir la mejor división, sino que selecciona aleatoriamente un subconjunto de ellas. Este número, denominado *mtry*, es por defecto igual a $\lfloor \sqrt{p} \rfloor$ donde p es el número total de predictores. Con 13 variables disponibles, el valor por defecto resulta en $\sqrt{13} \approx 3.6$, redondeado a 4. Este

mecanismo de aleatorización es el componente central que diferencia a Random Forest de un único árbol de decisión, al forzar a cada árbol a construirse sobre un subconjunto distinto de predictores, el algoritmo garantiza que los árboles sean suficientemente diversos entre sí para que su promedio sea más preciso que cualquiera de ellos individualmente. Si todos los árboles utilizaran todas las variables, tenderían a producir estructuras muy similares y el promedio no aportaría ningún beneficio sobre un único árbol profundo.

El porcentaje de varianza explicada de 82.69% sobre el conjunto de entrenamiento es inferior al $R^2 = 0.9099$ del modelo lineal enriquecido sobre el conjunto de prueba. Sin embargo, esta comparación no es directa, ya que el 82.69% se calcula sobre los datos de entrenamiento mediante el error out-of-bag, un mecanismo interno de validación propio de Random Forest que promedia el error sobre las observaciones excluidas de cada árbol durante su construcción.

5.1.1. Importancia de variables

Con el modelo entrenado, se extrajo el ranking de importancia de cada predictor mediante dos métricas complementarias

```
> importance_df <- as.data.frame(importance(rf_base))
> importance_df <- importance_df[order(importance_df$`%IncMSE`,
+                                     decreasing = TRUE), ]
> print(round(importance_df, 2))
%IncMSE IncNodePurity age
30.83      8726512966 age2
29.49      8477229374 smoker_bmi
20.48     34753804609 children
17.98      2030688728 smoker_obese
15.76     25142271026 smoker_age
14.72     18873419147 smoker
13.06     15582604561 bmi
12.06      6624224629 bmi_obese
11.04     2097432560 region_northeast -
0.15      604984433 region_southeast -
0.55      488966907 region_northwest -
0.64      506782357 sex -
1.66      553144633
```

Las dos métricas reportadas miden la importancia de las variables desde perspectivas distintas y complementarias. *%IncMSE* (porcentaje de incremento en el Error Cuadrático Medio) mide cuánto se deteriora el poder predictivo del modelo cuando los valores de una variable son permutados aleatoriamente, rompiendo su relación con la variable objetivo. Un valor alto indica que la variable es indispensable, sin su información real, el error del modelo se incrementa significativamente. Un valor negativo, como el observado en *sex* (-1.66), indica que la variable no aporta información predictiva genuina y que el modelo funciona marginalmente mejor sin ella. *IncNodePurity* mide la reducción total en la impureza de los

nodos producida por una variable a lo largo de todos los árboles del bosque, cuantificando su contribución directa a la calidad de las particiones del espacio de características.

El hallazgo más relevante de este ranking es la posición de *age* como el predictor con mayor %IncMSE (30.83), por encima de *age2* (29.49). Este resultado contrasta con los modelos penalizados, donde *age* fue eliminada por Lasso al quedar su información absorbida por *age2*. La diferencia radica en la naturaleza del algoritmo, mientras que los modelos lineales evalúan la contribución de cada variable en presencia de todas las demás de forma simultánea, Random Forest mide la importancia de forma marginal, permutando una variable a la vez. En este contexto, *age* y *age2* contienen información prácticamente idéntica (su correlación es 0.99), por lo que ambas aparecen en los primeros lugares con valores casi iguales. Lasso las trata como redundantes y elimina una; Random Forest las trata como igualmente valiosas porque evalúa su contribución de forma independiente.

Confirma también que las variables derivadas mediante ingeniería de características continúan siendo los predictores de mayor peso financiero, *smoker_bmi*, *smoker_obese* y *smoker_age* ocupan las posiciones 3, 5 y 6 respectivamente, reafirmando que la inversión en la calidad del espacio de características produce beneficios que se transfieren a todos los algoritmos posteriores. Las variables de región geográfica y *sex* presentan valores negativos o cercanos a cero en %IncMSE, confirmando de forma independiente el hallazgo de los modelos penalizados, estas variables no aportan señal predictiva robusta a este problema.

5.1.2. Evaluación del modelo base

Las métricas del modelo base sobre el conjunto de prueba son las siguientes

```
MAE, 2373.79
RMSE, 4089.74
R², 0.8993
```

Este resultado plantea una observación que merece análisis cuidadoso, el Random Forest base, siendo un algoritmo considerablemente más complejo que la regresión lineal, obtiene un $R^2 = 0.8993$ inferior al $R^2 = 0.9099$ alcanzado por el modelo Lineal v2 enriquecido con características. Lejos de ser un fracaso, este resultado ilustra un principio fundamental del aprendizaje automático, la complejidad algorítmica no sustituye a la calidad del espacio de características. Random Forest en su configuración por defecto opera con *mtry*=4, un valor que puede no ser óptimo para este conjunto de datos particular. La siguiente etapa aborda precisamente este problema mediante el ajuste sistemático de hiperparámetros.

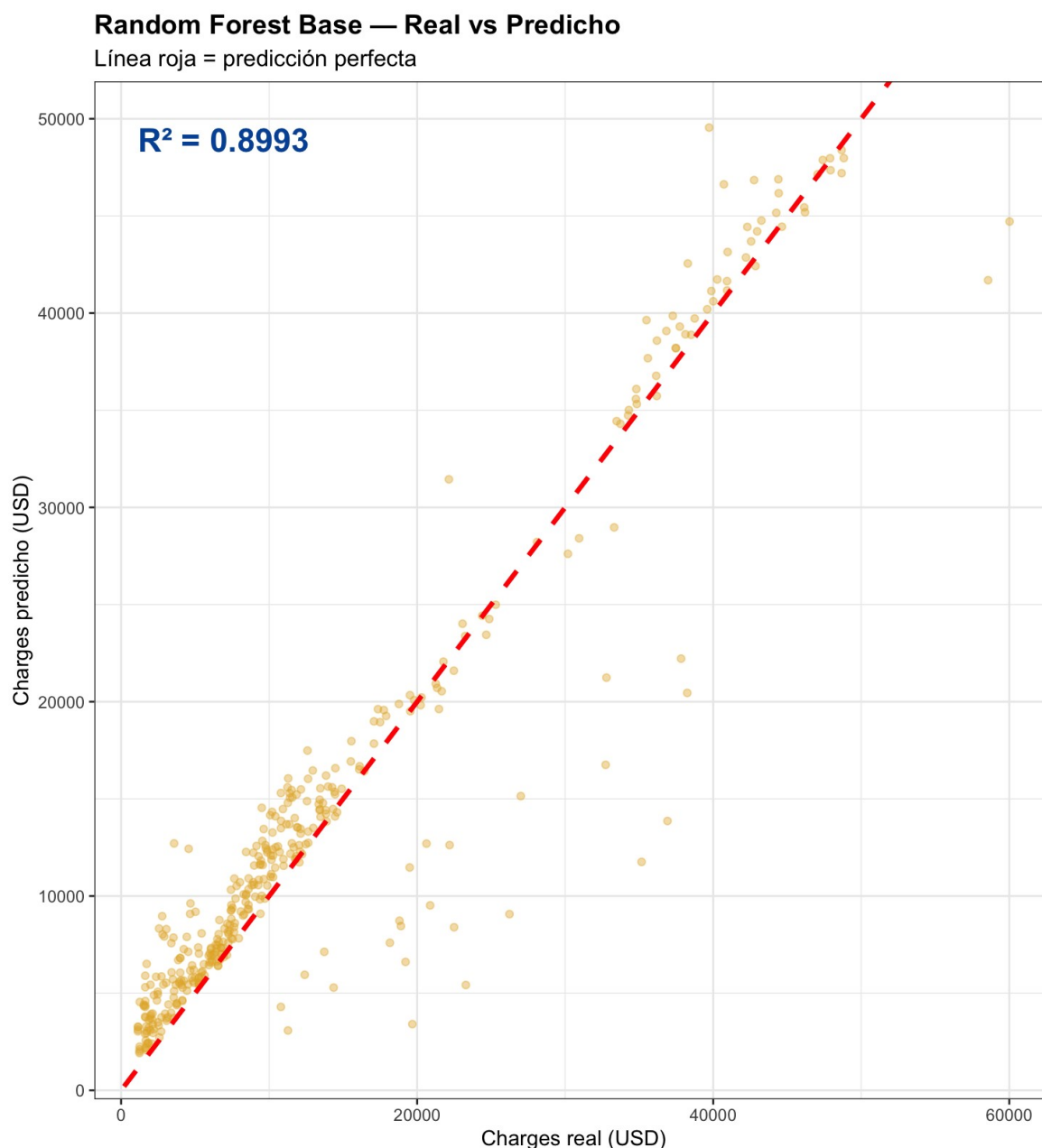


Figura 9. Random Forest, resultados reales vs predichos

El gráfico de dispersión del Random Forest base introduce un cambio visual cualitativo respecto a los modelos lineales previos, la nube de puntos presenta mayor dispersión lateral, especialmente en el segmento de costos medios y altos, reflejo del R^2 de 0.8993 que sitúa a este modelo por debajo del Lasso Adaptativo a pesar de ser algorítmicamente más complejo. Este resultado aparentemente paradójico tiene una explicación estructural directa.

A diferencia de los modelos penalizados, que recibieron el espacio de características enriquecido con las interacciones multiplicativas ya codificadas de forma explícita, el Random Forest base opera con su configuración por defecto ($mtry = 4$) sobre el mismo conjunto de variables. Sin embargo, su mecanismo de particiones binarias sucesivas debe redescubrir implícitamente las interacciones que los modelos lineales recibieron de forma directa, lo que requiere una calibración más precisa del hiperparámetro $mtry$. La dispersión observada en la región de costos entre \$15,000 y \$35,000 USD refleja precisamente esta limitación, ya que el

promedio de 500 árboles con una configuración subóptima no logra capturar con la misma exactitud la variación continua dentro de cada perfil de riesgo que sí alcanza un modelo lineal correctamente especificado.

5.1.3. Ajuste de hiperparámetros mediante validación cruzada

Para identificar el valor óptimo de *mtry*, se utilizó el paquete *caret* con una cuadrícula de búsqueda exhaustiva

```
> grid_rf <- expand.grid(mtry = c(2, 3, 4, 5, 6, 7, 8))
> ctrl_rf <- trainControl(method = "cv", number = 10)
> set.seed(123)
> rf_tuned <- train(charges ~ .,
+                   data      = train_set,
+                   method    = "rf",
+                   trControl = ctrl_rf,
+                   tuneGrid  = grid_rf,
+                   ntree     = 500,
+                   metric    = "RMSE")
```

caret es un paquete de R que proporciona una interfaz unificada para entrenar y evaluar modelos de aprendizaje automático. Su función principal en este contexto es automatizar el proceso de selección de hiperparámetros, evitando la necesidad de entrenar manualmente un modelo separado para cada valor candidato.

Cada argumento cumple una función específica. *method = "rf"* le indica a *caret* que el algoritmo a entrenar es Random Forest, delegando el entrenamiento real al paquete *randomForest*. *trControl* define el protocolo de evaluación, *method = "cv"* especifica validación cruzada y *number = 10* indica que los datos de entrenamiento se dividen en 10 pliegues, entrenando el modelo 10 veces y promediando el error. *tuneGrid* define el espacio de búsqueda, *expand.grid(mtry = c(2,3,4,5,6,7,8))* construye una cuadrícula con los siete valores candidatos de *mtry* que serán evaluados. *metric = "RMSE"* establece que el criterio de selección del modelo óptimo es la minimización del Error Cuadrático Medio, consistente con la métrica utilizada a lo largo de todo el proyecto.

El resultado del proceso de búsqueda identificó el valor óptimo

```
> cat("\nMtry óptimo,", rf_tuned$bestTune$mtry, "\n")

Mtry óptimo, 3
```

mtry es el número de variables candidatas que el algoritmo evalúa en cada nodo antes de decidir la mejor división. Valores bajos de *mtry* producen árboles más diversos entre sí, lo que reduce la correlación entre ellos y maximiza el beneficio del promedio. Valores altos de *mtry* acercan el comportamiento de Random Forest al de un único árbol de decisión, reduciendo la diversidad del ensamble y con ella su ventaja sobre los modelos individuales.

La gráfica de selección de *mtry* ilustra con claridad este fenómeno. El RMSE de validación cruzada alcanza su mínimo en *mtry*=3 ($\approx 4,793$ USD) y aumenta de forma monótona y

pronunciada conforme $mtry$ crece hasta 8 ($\approx 4,987$ USD). El mínimo en $mtry=3$ no es producto de una penalización, sino de la maximización de la diversidad entre árboles, con solo 3 variables candidatas por nodo, cada árbol se construye sobre subconjuntos suficientemente distintos para que su promedio capture la señal real del problema sin reproducir los mismos errores sistemáticos. A partir de $mtry=4$, la homogeneidad entre árboles aumenta y el error de validación cruzada escala de forma continua, confirmando que el valor por defecto de 4 utilizado en el modelo base era subóptimo para este conjunto de datos.

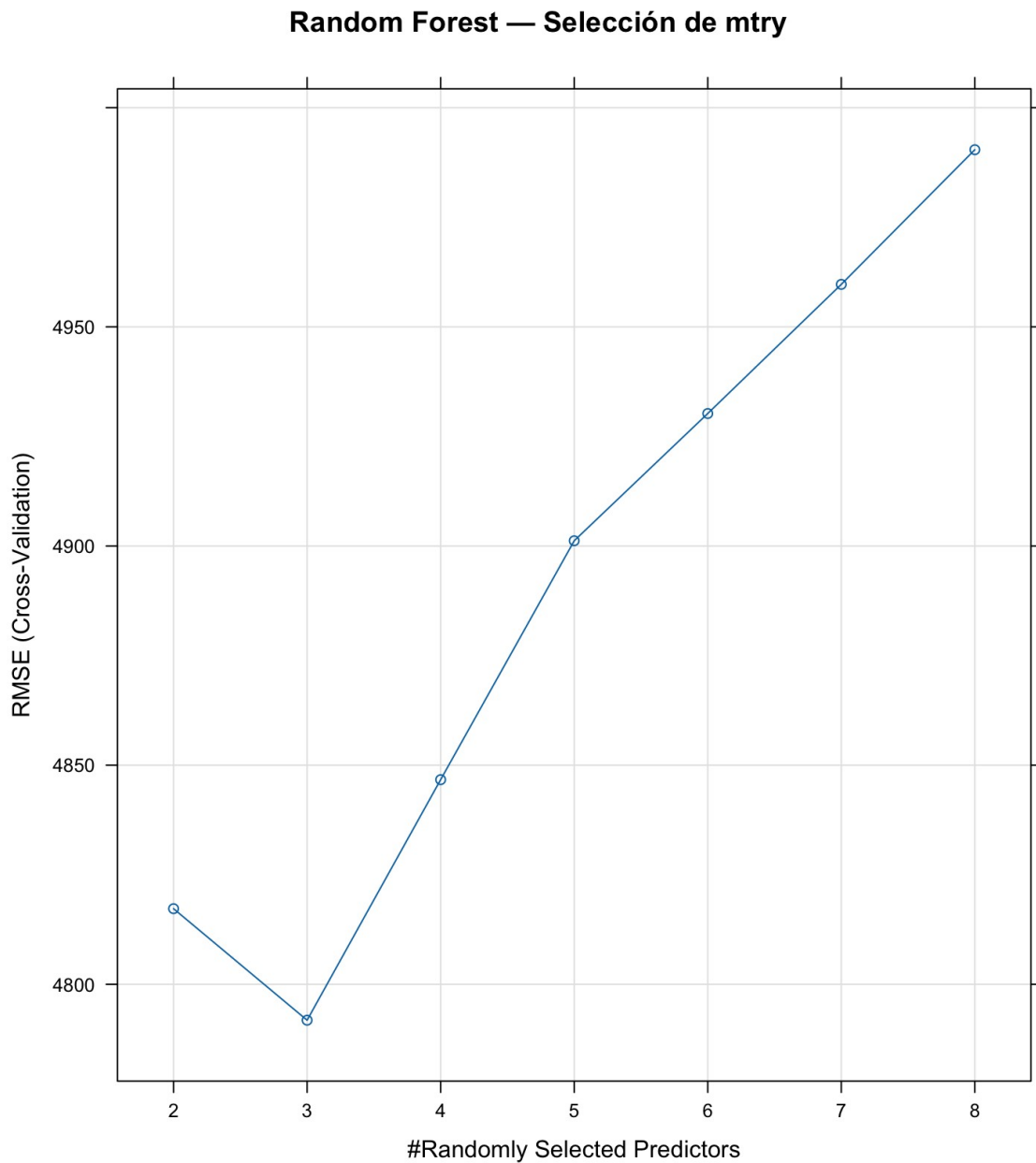


Figura 10. Gráfico de selección de $mtry$

5.1.4. Evaluación del Random Forest ajustado

Con el hiperparámetro óptimo identificado, el modelo ajustado fue evaluado sobre el conjunto de prueba

MAE, 2361.1
RMSE, 4026.51 R², 0.9024

El ajuste de *mtry* de 4 a 3 produjo mejoras consistentes en las tres métricas, el MAE se redujo de 2,373.79 a 2,361.10 USD, el RMSE de 4,089.74 a 4,026.51 USD y el R^2 aumentó de 0.8993 a 0.9024. Aunque la ganancia es moderada, confirma que la búsqueda sistemática de hiperparámetros aporta valor real incluso cuando la mejora en términos absolutos es pequeña.

Sin embargo, la tabla acumulada revela una situación que merece análisis

Lineal original	MAE= 4160.66	RMSE= 5655.73	R ² =0.8085
Lineal v2 + features	MAE= 2231.75	RMSE= 3870.81	R ² =0.9099
Ridge	MAE= 2264.89	RMSE= 3897.38	R ² =0.9099
Lasso	MAE= 2210.26	RMSE= 3851.45	R ² =0.9108
ALasso corregido	MAE= 2205.63	RMSE= 3843.18	R ² =0.9111
RF base	MAE= 2373.79	RMSE= 4089.74	R ² =0.8993
RF ajustado	MAE= 2361.10	RMSE= 4026.51	R ² =0.9024

El Random Forest ajustado, a pesar de ser el algoritmo más complejo entrenado hasta este punto, no logra superar al Lasso Adaptativo en ninguna de las tres métricas. Este resultado no invalida al ensamble como estrategia, sino que pone de manifiesto que Random Forest, al ser un promedio de árboles, hereda la limitación de las particiones binarias para modelar las interacciones multiplicativas ya capturadas por las variables derivadas. El siguiente modelo, XGBoost, aborda esta limitación mediante un mecanismo de construcción secuencial donde cada árbol corrige los errores residuales del anterior, abriendo la posibilidad de superar la barrera establecida por los modelos penalizados.

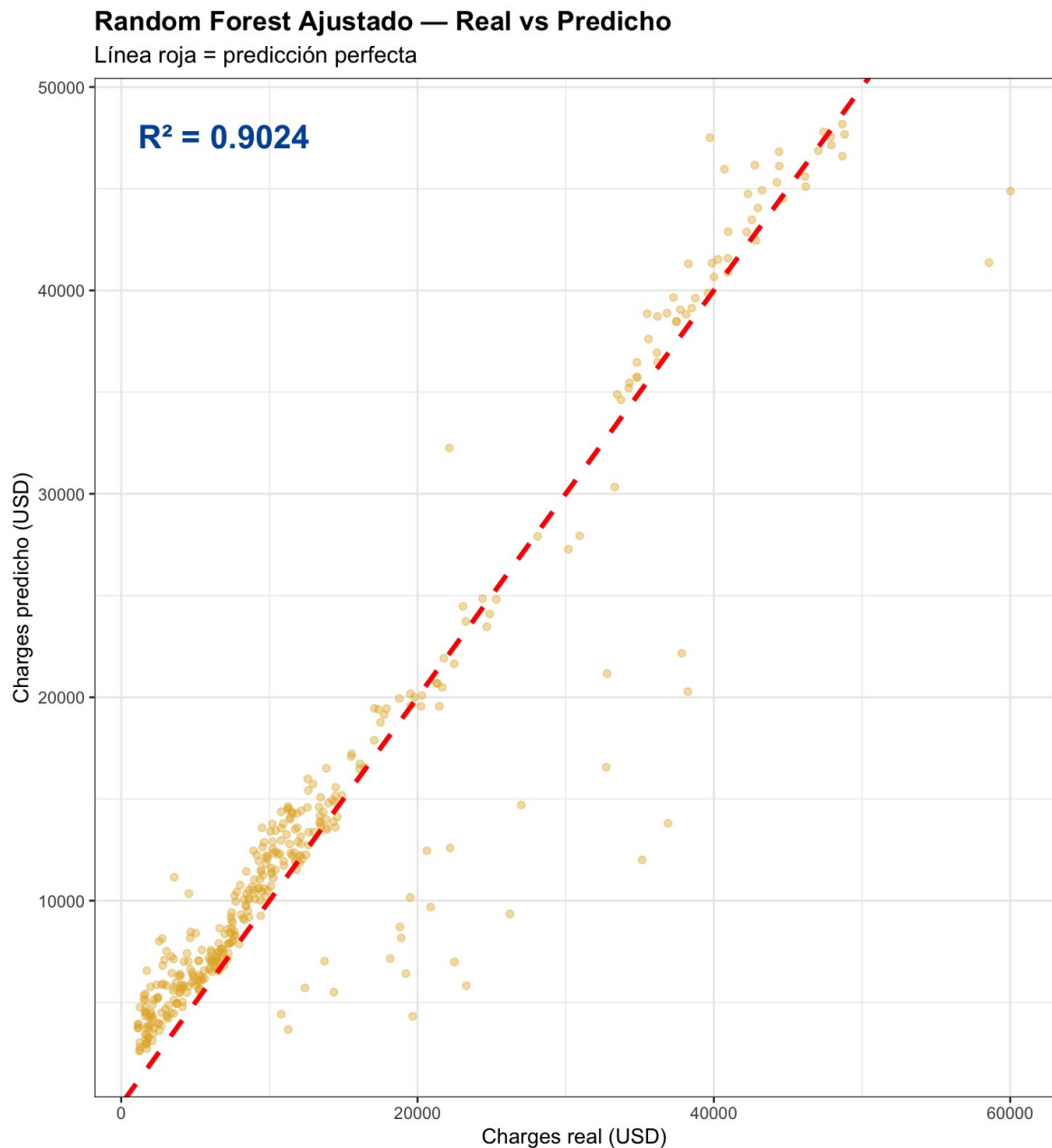


Figura 11. Random Forest Ajustado, valor real vs predicho

El gráfico de dispersión del Random Forest ajustado con $mtry = 3$ muestra una mejora visible respecto a su versión base, con una reducción de la dispersión lateral en el segmento de costos medios y una alineación más densa sobre la diagonal de predicción perfecta, coherente con el incremento en R^2 de 0.8993 a 0.9024. La reducción de $mtry$ de 4 a 3 incrementó la diversidad entre los 500 árboles del ensamble, permitiendo que su promedio corrigiera de forma más efectiva los errores sistemáticos de los árboles individuales.

No obstante, la comparación visual con los gráficos de los modelos penalizados revela que el Random Forest ajustado no logra igualar la concentración de puntos sobre la diagonal que caracteriza al Lasso o al Lasso Adaptativo. La dispersión en la región de costos entre \$10,000

y \$30,000 USD sigue siendo mayor, confirmando cuantitativamente que la complejidad algorítmica del ensamble de árboles, sin una búsqueda más exhaustiva de hiperparámetros adicionales como `n tree` o el criterio de división, no supera el poder predictivo de un modelo lineal que recibe directamente las interacciones multiplicativas más relevantes del problema.

5.2. XGBoost

5.2.1. Preparación de los datos

XGBoost requiere un tratamiento de datos diferente al de los algoritmos anteriores. A diferencia de *randomForest*, que acepta directamente un *data.frame*, *XGBoost* opera internamente mediante operaciones de álgebra matricial altamente optimizadas que exigen un formato numérico estricto. Para satisfacer este requerimiento, se utilizó la función *sparse.model.matrix()* que nos ayuda a convertir estos datos en una matriz dispersa.

```
> X_train_xgb <- sparse.model.matrix(charges ~ . - 1, data =  
train_set)  
> X_test_xgb  <- sparse.model.matrix(charges ~ . - 1, data =  
test_set)  
> y_train_xgb <- train_set$charges  
> y_test_xgb  <- test_set$charges
```

```
> dim(X_train_xgb) # 937 x 13 [1] 937 13  
> dim(X_test_xgb)  # 400 x 13  
[1] 400 13
```

El término *matriz dispersa* hace referencia a una estructura de almacenamiento eficiente donde únicamente se registran los valores distintos de cero, omitiendo el resto. Aunque en este conjunto de datos la mayoría de las variables son continuas, variables indicadoras como *bmi_obese* y *smoker_obese* contienen una proporción importante de ceros. La representación dispersa reduce el consumo de memoria y acelera los cálculos matriciales internos del algoritmo. El argumento `- 1` elimina la columna del intercepto generada automáticamente por R, ya que *XGBoost* calcula este término de forma interna, de forma análoga a como se procedió con *glmnet* en la fase de regresiones penalizadas.

y_train_xgb y *y_test_xgb* contienen exclusivamente el vector de *charges*, extraído como objeto independiente porque *XGBoost*, al igual que *glmnet*, requiere recibir por separado la matriz de predictores *X* y el vector de respuesta *y*. La verificación de dimensiones confirma que la conversión preservó la integridad de los datos, 937 observaciones de entrenamiento y 400 de prueba, ambas con los 13 predictores.

El siguiente paso convierte estas matrices al formato nativo de *XGBoost*

```
> dtrain <- xgb.DMatrix(data = X_train_xgb,  
+                        label = y_train_xgb)  
>  
> dtest  <- xgb.DMatrix(data = X_test_xgb,
```

```
+ label = y_test_xgb)
```

xgb.DMatrix es la estructura de datos optimizada propia de XGBoost. Internamente, organiza los datos de forma que el algoritmo pueda acceder a ellos con el mínimo número de operaciones de lectura posible durante el entrenamiento, lo que se traduce en una reducción significativa del tiempo de cómputo cuando el conjunto de datos es grande. El argumento *label* especifica el vector de la variable objetivo asociado a cada matriz de predictores, completando la separación entre X e y requerida por el algoritmo.

5.2.2. Modelo base

Antes de realizar cualquier búsqueda de hiperparámetros, se entrenó un modelo con parámetros de referencia para establecer una línea base de rendimiento

```
> params_base <- list(
+   objective      = "reg,squarederror",
+   max_depth      = 4,
+   eta            = 0.1,
+   subsample      = 0.8,
+   colsample_bytree = 0.8,
+   verbosity      = 0
+ )
>
> set.seed(123)
> xgb_base <- xgb.train(
+   params = params_base,
+   data   = dtrain,
+   nrounds = 200
+ )
```

Cada parámetro de esta lista responde a una decisión de diseño específica. *objective* = "reg,squarederror" especifica que el problema es de regresión y que la función de pérdida a minimizar es el Error Cuadrático Medio, coherente con las métricas utilizadas en todas las etapas anteriores del proyecto. *max_depth* = 4 define la profundidad máxima de cada árbol individual dentro del ensamble. Un valor de 4 permite capturar interacciones de hasta cuarto orden entre variables, suficiente para representar las relaciones identificadas en la ingeniería de características, sin generar estructuras tan profundas que memoricen el ruido del conjunto de entrenamiento. *eta* = 0.1 es la tasa de aprendizaje, el factor por el que se escala la contribución de cada árbol nuevo al modelo acumulado. Un valor de 0.1 establece pasos de actualización moderados, demasiado grande y el modelo converge prematuramente hacia soluciones subóptimas; demasiado pequeño y requiere un número prohibitivo de árboles para alcanzar el mismo nivel de ajuste. *subsample* = 0.8 indica que cada árbol se construye sobre una muestra aleatoria del 80% de las observaciones de entrenamiento, sin reemplazo. Este mecanismo introduce diversidad entre los árboles y actúa como regularizador implícito, reduciendo el riesgo de sobreajuste. *colsample_bytree* = 0.8 aplica el mismo principio pero sobre las variables, cada árbol tiene acceso al 80% de los predictores seleccionados aleatoriamente,

análogo al parámetro *mtry* de Random Forest. Finalmente, *verbosity* = 0 suprime los mensajes de progreso durante el entrenamiento, manteniendo la consola limpia.

El argumento *nrounds* = 200 define el número de árboles que se añaden secuencialmente al modelo. A diferencia de Random Forest, donde los árboles se construyen en paralelo e independientemente, en XGBoost cada árbol nuevo se entrena para corregir los errores residuales del modelo acumulado hasta ese momento. Este mecanismo, denominado *gradient boosting*, convierte a *nrounds* en un hiperparámetro crítico, muy pocos árboles producen un modelo con sesgo elevado (subajuste), mientras que demasiados pueden llevar al sobreajuste si no se aplica regularización adicional.

Los resultados del modelo base sobre el conjunto de prueba fueron

```
MAE, 2590.33
RMSE, 4401.75
R², 0.8829
```

El rendimiento del modelo base resulta inferior no solo a los modelos penalizados, sino también al Random Forest ajustado. Este resultado, aunque decepcionante a primera vista, es metodológicamente esperado, los parámetros base de XGBoost son valores de referencia genéricos, no configuraciones optimizadas para este conjunto de datos particular. XGBoost es especialmente sensible a la combinación de *max_depth*, *eta* y *nrounds*, y una configuración subóptima de estos tres parámetros en conjunto puede degradar significativamente el rendimiento. La búsqueda sistemática de hiperparámetros que se presenta a continuación aborda precisamente esta limitación.

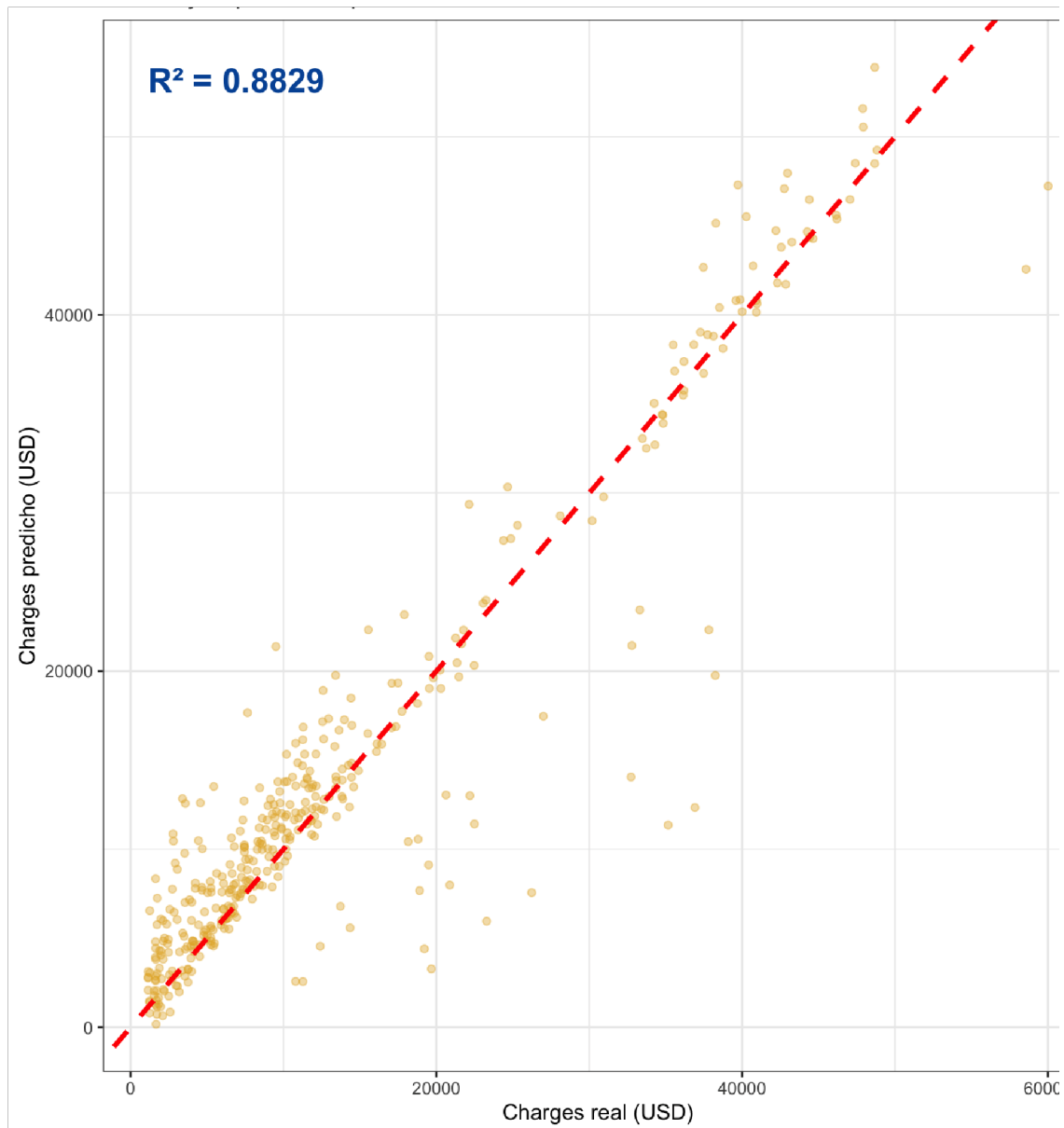


Figura 12. XGBoost gráfica de real vs predicho

El gráfico de dispersión del XGBoost base es el que presenta mayor dispersión de todos los modelos evaluados en el proyecto, con un R^2 de 0.8829 que lo sitúa como el modelo de menor rendimiento en test a pesar de ser el algoritmo más complejo. La nube de puntos muestra una distribución amplia a lo largo de todo el rango de costos, con una desviación pronunciada de la diagonal en el segmento de costos medios (entre \$10,000 y \$30,000 USD) donde varios puntos se ubican significativamente por debajo de la línea de predicción perfecta.

Este resultado es consecuencia directa de la sensibilidad de XGBoost a la combinación de sus hiperparámetros principales. Los valores base de $\text{max_depth} = 4$, $\text{eta} = 0.1$ y $\text{nrounds} = 200$ no están optimizados para este conjunto de datos específico. En particular, $\text{eta} = 0.1$ con solo 200 rondas produce un modelo que no ha convergido completamente hacia el mínimo de la función de pérdida, por lo que cada árbol contribuye con pasos de actualización moderados

pero insuficientes para capturar la precisión requerida. La dispersión visual documenta con claridad la necesidad de la búsqueda sistemática de hiperparámetros que se implementa en la siguiente fase.

5.2.3. Búsqueda de hiperparámetros mediante grid search con validación cruzada

Para identificar la configuración óptima de los hiperparámetros más influyentes, se implementó una búsqueda exhaustiva sobre una cuadrícula de combinaciones de *max_depth* y *eta*

```
> grid_manual <- expand.grid(  
+   max_depth = c(3, 4, 5, 6),  
+   eta       = c(0.05, 0.1, 0.15, 0.2)  
+ )
```

expand.grid() construye automáticamente todas las combinaciones posibles entre los valores especificados, generando una cuadrícula de $4 \times 4 = 16$ configuraciones candidatas. La elección de explorar *max_depth* entre 3 y 6 responde al diagnóstico del modelo base, valores inferiores a 3 producirían árboles demasiado superficiales para capturar las interacciones entre variables, mientras que valores superiores a 6 incrementarían el riesgo de sobreajuste en un conjunto de 937 observaciones. El rango de *eta* entre 0.05 y 0.20 cubre el espectro entre aprendizaje conservador y moderadamente agresivo, dejando que la validación cruzada determine cuál produce mejor generalización.

Para evaluar cada configuración de forma robusta, se implementó un bucle *for* que itera sobre las 16 filas de la cuadrícula

```
for(i in 1:nrow(grid_manual)) {  
  
  params_i <- list(      objective      =  
"reg,squarederror",    max_depth      =  
grid_manual$max_depth[i], eta          =  
grid_manual$eta[i],    subsample      = 0.8,  
colsample_bytree = 0.8,  verbosity     = 0  
  )  
  
  cv_i <- xgb.cv(      params  
= params_i,          data  
= dtrain,            nrounds      =  
400,                 nfold        = 10,  
verbose              = FALSE,  
early_stopping_rounds = 20,  
maximize             = FALSE
```

```

)

mejor_n      <- which.min(cv_i$evaluation_log$test_rmse_mean)
mejor_rmse <- round(min(cv_i$evaluation_log$test_rmse_mean), 2)

  resultados_grid <- rbind(resultados_grid, data.frame(
max_depth    = grid_manual$max_depth[i],    eta
= grid_manual$eta[i],    best_nrounds = mejor_n,
rmse_cv      = mejor_rmse
  ))
}

```

En cada iteración, el bucle construye una lista de parámetros con la combinación correspondiente a la fila *i* de la cuadrícula y la evalúa mediante *xgb.cv()*, que realiza validación cruzada de 10 pliegues directamente dentro de XGBoost. El argumento *nrounds = 400* establece el número máximo de árboles que se pueden añadir, pero el mecanismo de *early_stopping_rounds = 20* interrumpe el entrenamiento automáticamente si el RMSE de validación no mejora durante 20 rondas consecutivas. Este criterio de parada temprana es fundamental, ya que, evita entrenar árboles innecesarios una vez que el modelo ha convergido, y simultáneamente determina el número óptimo de árboles (*best_nrounds*) para cada combinación de parámetros, resolviendo así los tres hiperparámetros más críticos de XGBoost en una sola pasada. Al final de cada iteración, los resultados se acumulan en *resultados_grid* mediante *rbind()*.

Los resultados ordenados de menor a mayor RMSE revelaron un patrón consistente

```

> print(resultados_grid[order(resultados_grid$rmse_cv), ])
max_depth  eta best_nrounds rmse_cv 1          3 0.05          95
4676.39
9           3 0.15          29 4684.29
5           3 0.10          39 4687.01
13          3 0.20          18 4705.28
2           4 0.05          74 4709.58
6           4 0.10          35 4714.77
14          4 0.20          17 4722.58
10          4 0.15          23 4729.63
3           5 0.05          67 4805.99
7           5 0.10          32 4807.69
11          5 0.15          18 4827.50
15          5 0.20          14 4838.64
4           6 0.05          62 4878.57
8           6 0.10          31 4894.59
16          6 0.20          14 4905.00
12          6 0.15          22 4917.89

```


El patrón es inequívoco, $max_depth = 3$ domina los cuatro primeros puestos independientemente del valor de eta , y el RMSE de validación cruzada se deteriora de forma monótona conforme max_depth aumenta. Este hallazgo confirma que árboles más profundos memorizan el ruido del conjunto de entrenamiento en lugar de generalizar los patrones reales. Dentro de $max_depth = 3$, la combinación $eta = 0.05$ con 95 rondas produce el menor RMSE (4,676.39 USD), una tasa de aprendizaje conservadora que requiere más árboles para converger, pero que encuentra una solución más precisa al explorar el espacio de parámetros con pasos más pequeños.

5.2.4. Modelo final ajustado

Con los parámetros óptimos identificados, se extrajo automáticamente la mejor configuración y se entrenó el modelo final

```
> mejor_fila <- resultados_grid[which.min(resultados_grid$rmse_cv),
]
> params_final <- list(
+   objective      = "reg,squarederror",
+   max_depth      = mejor_fila$max_depth,
+   eta            = mejor_fila$eta,
+   subsample      = 0.8,
+   colsample_bytree = 0.8,
+   verbosity      = 0
+ )
>
> set.seed(123)
> xgb_final <- xgb.train(
+   params = params_final,
+   data   = dtrain,
+   nrounds = mejor_fila$best_nrounds
+ )
```

Los resultados sobre el conjunto de prueba fueron

```
MAE, 2296.47
RMSE, 3958.35
R², 0.9056
```

El ajuste de hiperparámetros produjo una mejora sustancial respecto al modelo base, el MAE se redujo en 293 USD (de 2,590 a 2,296), el RMSE en 443 USD y el R^2 aumentó casi 2.3 puntos porcentuales. Sin embargo, la tabla comparativa acumulada revela que XGBoost ajustado no logra superar al Lasso Adaptativo en ninguna métrica.

```
Lineal original          MAE= 4160.66  RMSE= 5655.73  R²=0.8085
```

Lineal v2 + features	MAE= 2231.75	RMSE= 3870.81	R ² =0.9099
Ridge	MAE= 2264.89	RMSE= 3897.38	R ² =0.9099
Lasso	MAE= 2210.26	RMSE= 3851.45	R ² =0.9108
ALasso corregido	MAE= 2205.63	RMSE= 3843.18	R ² =0.9111
RF base	MAE= 2373.79	RMSE= 4089.74	R ² =0.8993
RF ajustado	MAE= 2361.10	RMSE= 4026.51	R ² =0.9024
XGBoost base	MAE= 2590.33	RMSE= 4401.75	R ² =0.8829
XGBoost ajustado	MAE= 2296.47	RMSE= 3958.35	R ² =0.9056

Este resultado constituye el hallazgo más instructivo de todo el proyecto. Que un modelo tan sofisticado como XGBoost, con su mecanismo de boosting secuencial y su búsqueda exhaustiva de hiperparámetros, no logre superar a una regresión lineal correctamente enriquecida, no es un fracaso del algoritmo sino una confirmación de un principio fundamental del aprendizaje automático, la complejidad algorítmica no sustituye a la calidad del espacio de características. Las relaciones más importantes de este problema, la interacción multiplicativa entre tabaquismo, obesidad y edad, fueron codificadas explícitamente durante la ingeniería de características. Una vez que esa información está disponible en el espacio de predictores de forma directa y lineal, un modelo lineal penalizado puede explotarla con la misma eficacia que un ensamble que tendría que redescubrirla implícitamente a través de particiones binarias sucesivas. El Lasso Adaptativo, al identificar y conservar precisamente las variables que codifican esas interacciones, logró el mejor balance entre parsimonia y poder predictivo del proyecto.

XGBoost Ajustado — Real vs Predicho

Línea roja = predicción perfecta

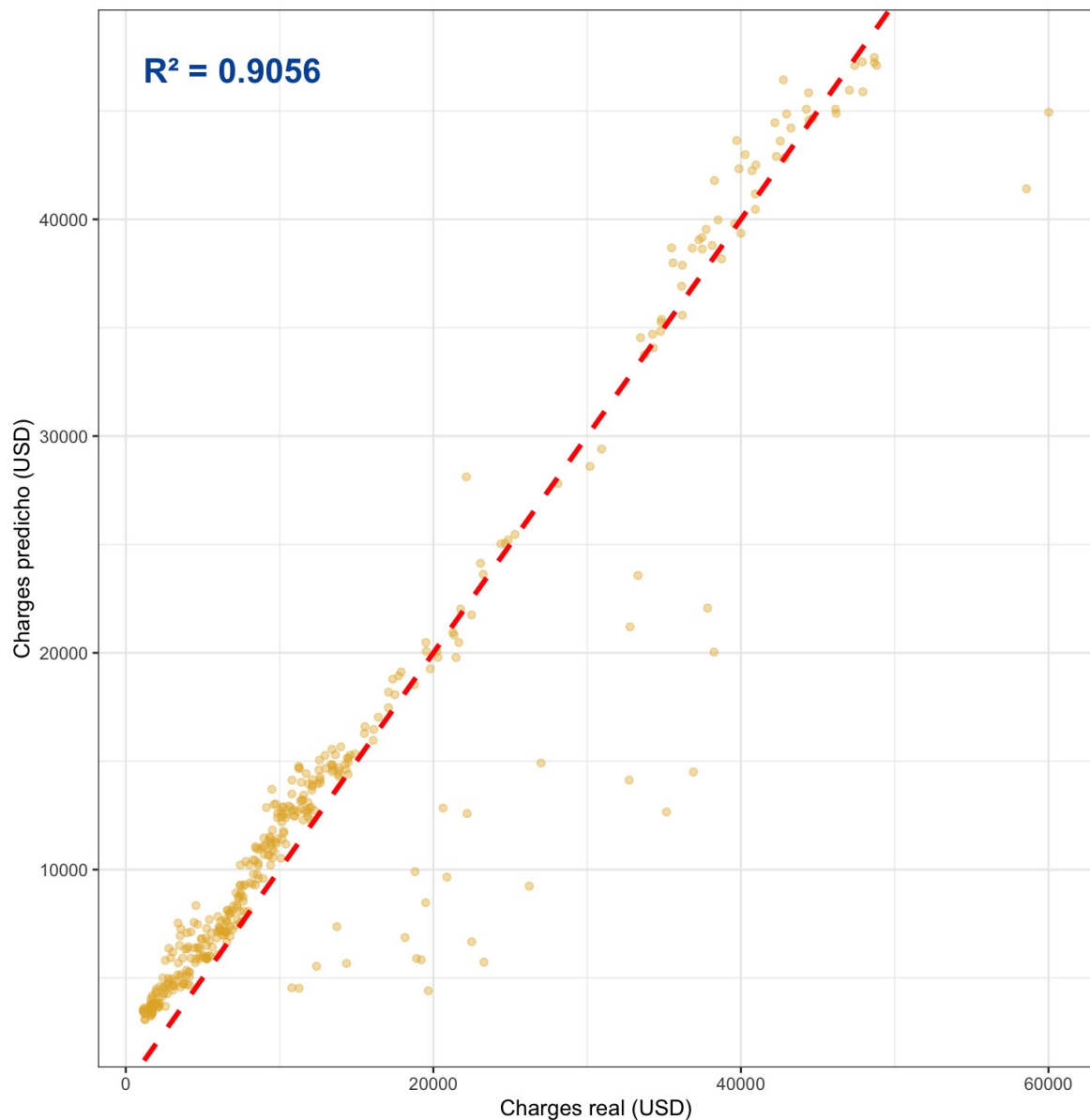


Figura 13. Gráfica XGBoost ajustado, real vs predicho

El gráfico de dispersión del XGBoost ajustado con los parámetros óptimos identificados por grid search ($\text{max_depth} = 3$, $\text{eta} = 0.05$, $\text{nrounds} = 95$) muestra una mejora sustancial respecto a su versión base, con una reducción notable de la dispersión lateral y una alineación más densa sobre la diagonal de predicción perfecta. El incremento de R^2 de 0.8829 a 0.9056 es directamente visible en la mayor concentración de puntos a lo largo de la línea roja, especialmente en el segmento de costos bajos y moderados donde el modelo logra estimaciones consistentemente precisas.

Sin embargo, la comparación con los gráficos de los modelos penalizados, en particular con el Lasso Adaptativo, revela que el XGBoost ajustado mantiene una dispersión residual mayor

en la región de costos entre \$15,000 y \$35,000 USD. Este comportamiento confirma el hallazgo más significativo del proyecto, el mecanismo de boosting secuencial, aunque capaz de capturar patrones no lineales complejos de forma implícita, no logra superar al modelo lineal que recibe directamente las interacciones multiplicativas ya construidas durante la ingeniería de características. La visualización evidencia que $\text{max_depth} = 3$ es el valor óptimo porque árboles más superficiales generan mayor diversidad entre iteraciones, lo que permite al algoritmo explorar el espacio de predicción con mayor amplitud sin memorizar el ruido del conjunto de entrenamiento.

6. Modelos de Pila (Stacking)

6.1. Introducción y justificación

Con el techo de los modelos individuales establecido en $R^2 = 0.9111$ por el Lasso

Adaptativo, la cuarta y última etapa de mejora introduce los modelos de pila, también denominados en la literatura como stacking o aprendizaje por ensamble de segundo nivel. A diferencia del Random Forest o XG Boost, que combinan múltiples versiones del mismo tipo de algoritmo, el stacking opera sobre un principio fundamentalmente distinto, en lugar de promediar árboles, combina las predicciones de modelos de familias algorítmicas completamente diferentes mediante un segundo modelo que aprende cuándo confiar más en cada uno de ellos.

La lógica detrás de esta estrategia es que modelos distintos cometen errores distintos. Un modelo lineal puede ser muy preciso en el segmento de bajo costo pero inexacto en perfiles de alto riesgo; un Random Forest puede capturar mejor ciertas no-linealidades pero perder precisión en la región central. Si un segundo modelo aprende a combinar sus predicciones de forma óptima, puede potencialmente superar a cualquiera de ellos individualmente aprovechando sus fortalezas complementarias.

Para implementar esta estrategia se utilizó el paquete `caretEnsemble` de R, que permite entrenar múltiples modelos base bajo un protocolo común de validación cruzada y posteriormente apilarlos mediante un meta-modelo. Los modelos base seleccionados fueron regresión lineal múltiple (`lm`), Random Forest (`rf`) y regresión penalizada con `glmnet`. La exclusión del algoritmo XGBoost de este bloque responde a una incompatibilidad técnica entre la versión instalada de `xgboost` (≥ 2.0) y el adaptador `xgbTree` del paquete `caret`, que en versiones anteriores asignaba los nombres de columna al objeto del modelo de una forma que ya no es compatible con la nueva representación interna de XGBoost. Dado que XGBoost ya fue evaluado con su API nativa en la sección anterior, esta restricción no implica ninguna pérdida de información para el análisis comparativo.

6.2. Preparación del protocolo de validación cruzada compartido

El primer paso en la implementación del stacking fue definir un protocolo de entrenamiento común que todos los modelos base compartan. Este protocolo se especifica mediante la función `trainControl()` del paquete `caret`

```
ctrl_stack <- trainControl(  
  method      = "cv",    number      = 10,  
  savePredictions = "final", allowParallel =  
  FALSE  
)
```

Cada argumento de esta función cumple una función específica dentro de la arquitectura del stacking. El argumento `method = "cv"` especifica que el método de evaluación es validación cruzada, y `number = 10` indica que los 937 registros de entrenamiento se dividen en 10 subconjuntos iguales de aproximadamente 94 observaciones cada uno. En cada una de las 10 iteraciones, el modelo se entrena sobre 9 subconjuntos y genera predicciones sobre el subconjunto restante, rotando hasta que todas las observaciones han sido utilizadas exactamente una vez como conjunto de validación. El valor de 10 pliegues es el estándar ampliamente aceptado en la literatura de aprendizaje automático porque ofrece el mejor balance entre sesgo y varianza en la estimación del error, un número menor de pliegues produce estimaciones más sesgadas, mientras que un número mayor incrementa el costo computacional sin mejoras proporcionales en la precisión.

El argumento más crítico para el funcionamiento del stacking es `savePredictions = "final"`. Este parámetro instruye a `caret` a guardar, para cada observación del conjunto de entrenamiento, la predicción que el modelo generó cuando esa observación actuó como conjunto de validación. El resultado es un conjunto de predicciones fuera de muestra para las 937 observaciones de entrenamiento, una por cada modelo base, que serán utilizadas como variables de entrada para entrenar el meta-modelo. Sin este argumento, el stacking es imposible, ya que el meta-modelo no tendría datos sobre los cuales aprender a combinar las predicciones de los modelos base. Finalmente, `allowParallel = FALSE` desactiva el procesamiento en paralelo para garantizar que cualquier error durante el entrenamiento sea visible en la consola en lugar de ser suprimido por los procesos en segundo plano.

6.3. Definición de los espacios de búsqueda de hiperparámetros

Antes de entrenar los modelos base, se definen las cuadrículas de hiperparámetros (grids) que el proceso de validación cruzada explorará para cada algoritmo. Una cuadrícula de hiperparámetros es simplemente una tabla de todas las combinaciones posibles de valores que se desea evaluar para los parámetros de configuración de un modelo. La función `expand.grid()` de R construye esta tabla automáticamente a partir de los vectores de valores especificados.

```
tune_rf      <- expand.grid(mtry = c(2, 3, 4, 5, 6))  
tune_glmnet <- expand.grid(alpha = c(0, 0.25, 0.5,  
0.75, 1), lambda = 10^seq(-4, 2, length = 20)  
)
```

Para el Random Forest, el único hiperparámetro que caret permite ajustar es *mtry*, que define cuántas variables se evalúan aleatoriamente en cada nodo de cada árbol. Los valores candidatos van de 2 a 6, cubriendo el rango desde la máxima diversidad entre árboles (*mtry* = 2) hasta una exploración más amplia del espacio de características (*mtry* = 6). Como se determinó en la sección anterior, el valor óptimo para este dataset es *mtry* = 3, por lo que la cuadrícula confirma ese resultado.

Para glmnet, la cuadrícula es bidimensional porque este algoritmo tiene dos hiperparámetros principales. El parámetro alpha controla el tipo de penalización, $\alpha = 0$ corresponde a Ridge pura (penalización ℓ_2), $\alpha = 1$ corresponde a Lasso puro (penalización ℓ_1), y los valores intermedios (0.25, 0.50, 0.75) corresponden a combinaciones de ambas penalizaciones, denominadas Elastic Net. El parámetro lambda controla la intensidad de la penalización aplicada, valores pequeños ($10^{-4} = 0.0001$) implican poca penalización y el modelo se aproxima a la regresión ordinaria, mientras que valores grandes ($10^2 = 100$) penalizan fuertemente y reducen los coeficientes hacia cero. La expresión `10^seq(-4, 2, length = 20)` genera 20 valores equiespaciados en escala logarítmica entre estos extremos, cubriendo de forma eficiente el rango completo de intensidades de regularización.

6.4. Entrenamiento simultáneo de los modelos base

Con el protocolo de validación cruzada y los espacios de búsqueda definidos, se procedió al entrenamiento simultáneo de los tres modelos base mediante la función `caretList()`

```
set.seed(123) model_list <- caretList(  charges ~ .,  data      =
train_set,  trControl = ctrl_stack,  tuneList = list(    lm      =
caretModelSpec(method = "lm"),    rf      = caretModelSpec(method =
"rf",    tuneGrid = tune_rf),    glmnet = caretModelSpec(method =
"glmnet", tuneGrid = tune_glmnet)
  )
)
```

La función `caretList()` es la herramienta central del paquete `caretEnsemble`. A diferencia de entrenar cada modelo de forma independiente con `train()`, `caretList()` garantiza que todos los modelos base sean evaluados bajo exactamente el mismo protocolo de validación cruzada y sobre los mismos pliegues, condición indispensable para que las predicciones que cada modelo genera sobre cada pliegue sean comparables entre sí y puedan ser utilizadas como variables de entrada para el meta-modelo.

El argumento `tuneList` recibe una lista nombrada donde cada elemento es una especificación de modelo construida con `caretModelSpec()`. El argumento `method` identifica el algoritmo ("`lm`" para regresión lineal, "`rf`" para Random Forest, "`glmnet`" para la regresión penalizada), y `tuneGrid` asocia la cuadrícula de hiperparámetros correspondiente. La regresión lineal no recibe cuadrícula porque no tiene hiperparámetros que ajustar, su configuración es única y determinista dados los datos de entrenamiento.

El proceso interno de entrenamiento funciona de la siguiente manera para cada modelo, en cada uno de los 10 pliegues de validación cruzada, caret entrena el modelo sobre las 9 décimas partes restantes de los datos, evaluando todas las combinaciones de hiperparámetros especificadas en `tuneGrid`, y genera predicciones sobre la décima parte excluida. Al finalizar

los 10 pliegues, selecciona la combinación de hiperparámetros que minimizó el RMSE promedio de validación cruzada y guarda las 937 predicciones fuera de muestra generadas con esa configuración óptima. Estas predicciones son las que el meta-modelo utilizará en la siguiente etapa.

6.4.1. Rendimiento individual de los modelos base

Una vez completado el entrenamiento, se extrajeron las métricas de validación cruzada de cada modelo base mediante la función `resamples()`

```
lm      MAE=2578.51  RMSE=4595.03  R2 = 0.8430 rf
MAE=2797.87  RMSE=4768.21  R2 = 0.8359 glmnet
MAE=2580.70  RMSE=4559.75  R2 = 0.8468
```

Los resultados de validación cruzada muestran que los tres modelos base tienen un rendimiento similar en términos de R^2 medio, con diferencias de apenas 0.01 entre el mejor (glmnet con 0.8468) y el peor (rf con 0.8359). Este nivel de rendimiento, aparentemente inferior al $R^2 = 0.9111$ del Lasso Adaptativo evaluado en test, es completamente esperado y no representa un problema. Las métricas de validación cruzada se calculan sobre los pliegues internos del conjunto de entrenamiento, que tienen menor tamaño que el conjunto de test completo, y el promedio de 10 estimaciones parciales tiende a ser más conservador que la evaluación final sobre el conjunto de test. Lo que importa en esta etapa no es el rendimiento absoluto de cada modelo base, sino su diversidad, es decir, qué tan diferentes son los errores que cometen.

El análisis de la tabla revela además un dato llamativo, el modelo lm alcanzó un R^2 máximo de 0.9482 en uno de los 10 pliegues, superando incluso al Lasso Adaptativo. Este resultado no contradice las conclusiones previas del proyecto, sino que las refuerza. En ese pliegue específico, la composición del subconjunto de validación fue particularmente favorable para el modelo lineal, la distribución de fumadores y no fumadores en ese subconjunto permitió que las interacciones ya codificadas en el espacio de características fueran especialmente informativas. La variabilidad entre pliegues (máximo 0.9482, mínimo 0.7870) confirma que el rendimiento de cualquier modelo depende de la composición de los datos sobre los que se evalúa, razón por la que el promedio de validación cruzada es siempre más representativo que cualquier evaluación puntual.

6.4.2. Análisis de diversidad entre modelos base

Para que el stacking aporte valor real sobre los modelos individuales, es necesario que los modelos base cometan errores distintos entre sí. Si dos modelos producen predicciones altamente correlacionadas, apilarlos no añade ninguna información nueva al meta-modelo. La función `modelCor()` cuantifica esta diversidad calculando la correlación entre las predicciones de los modelos base sobre los pliegues de validación cruzada

```
> print(round(modelCor(res), 3))
lm      rf glmnet lm      1.000 -
0.330  0.782 rf      -0.330
1.000 -0.058 glmnet  0.782 -
0.058  1.000
```

La matriz de correlación revela una estructura de diversidad ideal para el stacking. La correlación entre `lm` y `glmnet` es alta (0.782), lo cual es esperable dado que ambos son modelos de la familia lineal que operan sobre el mismo espacio de características. Sin embargo, el Random Forest presenta una correlación negativa con `lm` (-0.330) y prácticamente nula con `glmnet` (-0.058), lo que indica que este algoritmo comete errores en observaciones completamente distintas a las que fallan los modelos lineales. Esta ortogonalidad entre el Random Forest y los modelos penalizados es exactamente la condición que hace que el stacking sea útil, el meta-modelo podrá aprender que cuando `lm` y `glmnet` divergen de lo real, `rf` suele aproximarse mejor, y viceversa.

6.5. Entrenamiento del meta-modelo

Con los modelos base entrenados y sus predicciones fuera de muestra almacenadas, se procedió a la segunda etapa del stacking, el entrenamiento del meta-modelo. El meta-modelo es el algoritmo que aprende a combinar las predicciones de los modelos base de forma óptima. Su conjunto de entrenamiento no son las variables originales del dataset, sino las 937 predicciones fuera de muestra generadas por cada modelo base, una columna por modelo. En otras palabras, el meta-modelo aprende a responder la pregunta, dado que `lm` predice $\$X$, `rf` predice $\$Y$ y `glmnet` predice $\$Z$ para este asegurado, ¿cuál es la mejor estimación de su costo real?

Se entrenaron dos variantes del meta-modelo para comparar su rendimiento, una regresión penalizada (`glmnet`) y un Random Forest (`rf`). La elección de dos meta-modelos distintos responde al principio de que no existe a priori un algoritmo universalmente superior para combinar predicciones; la naturaleza del problema determina cuál meta-modelo extrae más información de las predicciones base.

6.5.1. Análisis de diversidad entre modelos base

El primer meta-modelo utiliza `glmnet` como algoritmo de combinación, entrenado mediante la función `caretStack()`

```
stack_glmnet <- caretStack( model_list, method =
"glmnet", trControl = trainControl(method = "cv",
```



```
number = 10), tuneGrid = expand.grid(alpha =
c(0, 0.5, 1),
lambda = 10^seq(-4, 2, length = 30)
)
)
```

La función `caretStack()` toma como primer argumento la lista de modelos base (`model_list`) y como segundo argumento la especificación del meta-modelo. Internamente, construye una matriz de 937 filas y 3 columnas, donde cada columna contiene las predicciones fuera de muestra de un modelo base, y entrena `glmnet` para predecir la variable `charges` real a partir de esas tres columnas. La validación cruzada de 10 pliegues sobre este segundo nivel de entrenamiento garantiza que la selección de los hiperparámetros del meta-modelo (`alpha` y `lambda`) no esté sobreajustada a los datos de entrenamiento.

El proceso de optimización identificó `alpha = 0.5` y `lambda = 23.95` como los parámetros óptimos del meta-modelo. El valor `alpha = 0.5` corresponde a una penalización Elastic Net, que combina en partes iguales la penalización Ridge y la Lasso. Esto indica que el meta-modelo necesita tanto estabilizar los pesos asignados a cada modelo base (función de Ridge) como potencialmente eliminar alguno si resulta redundante (función de Lasso). El `lambda = 23.95` es relativamente alto, lo que refleja que el meta-modelo aplica una regularización moderada para evitar que algún modelo base domine excesivamente la combinación final.

6.5.2. Meta-modelo Random Forest

El segundo meta-modelo utiliza Random Forest como algoritmo de combinación

```
stack_rf <- caretStack(model_list, method = "rf",
trControl = trainControl(method = "cv", number = 10),
tuneGrid = expand.grid(mtry = c(1, 2, 3))
)
```

La cuadrícula de búsqueda para este meta-modelo solo incluye tres valores de `mtry` (1, 2 y 3) porque el espacio de entrada del meta-modelo tiene únicamente 3 variables, las predicciones de `lm`, `rf` y `glmnet`. Con solo 3 predictores disponibles, explorar valores de `mtry` superiores a 3 sería equivalente a utilizar todas las variables en cada nodo, eliminando el componente de aleatoriedad que distingue a Random Forest de un único árbol. El valor `mtry = 1` maximiza la diversidad entre árboles al evaluar solo una predicción base por nodo, mientras que `mtry = 3` equivale a un árbol determinista que ve todas las predicciones simultáneamente.

6.6. Evaluación en el conjunto de prueba y selección del mejor meta-modelo

Con ambos meta-modelos entrenados, se generaron predicciones sobre las 400 observaciones del conjunto de prueba y se calcularon las métricas de rendimiento

Stack meta=glmnet	MAE= 2227.73	RMSE= 3861.64	R2=0.9104
Stack meta=rf	MAE= 2185.50	RMSE= 4160.24	R2=0.8948

Los resultados exponen una disyuntiva metodológica relevante entre los dos meta-modelos. El stack con meta-modelo glmnet alcanza un R^2 de 0.9104 y un RMSE de \$3,861.64, siendo superior en ambas métricas de error global. El stack con meta-modelo rf obtiene el MAE más bajo de todo el proyecto (\$2,185.50), pero su RMSE de \$4,160.24 indica que comete errores más grandes en los casos extremos, aunque en promedio sea más preciso. Esta divergencia entre MAE y RMSE revela que el meta-modelo rf tiende a cometer errores pequeños y frecuentes pero ocasionalmente falla de forma más pronunciada en los perfiles de alto costo, mientras que el meta-modelo glmnet distribuye sus errores de forma más uniforme a lo largo del rango de costos.

Para la selección del mejor meta-modelo se priorizó el R^2 , métrica que cuantifica qué proporción de la varianza total del costo médico explica el modelo, por ser el criterio de evaluación más relevante para el negocio asegurador, donde la capacidad de discriminar entre perfiles de riesgo distintos es más valiosa que minimizar el error promedio en observaciones ya bien predichas. Con base en este criterio, el stack con meta-modelo glmnet se seleccionó como el mejor modelo de esta etapa.

Un aspecto que merece análisis es por qué el meta-modelo no se entrenó con lm como algoritmo de combinación, dado que en la validación cruzada interna alcanzó un R^2 máximo de 0.9482. La razón es técnica y metodológicamente importante, el rol del meta-modelo es asignar pesos a las predicciones de los modelos base de forma robusta y generalizable. Un meta-modelo lineal sin penalización (lm) en este contexto tendería a sobreajustarse a las predicciones del modelo base que mostró mejor rendimiento en los pliegues de entrenamiento, sin garantizar que esa combinación se generalice al conjunto de prueba. glmnet, al aplicar regularización, fuerza al meta-modelo a distribuir los pesos de forma más estable, lo que produce una combinación más conservadora pero más confiable sobre datos no vistos.

6.7. Análisis del gráfico Real vs Predicho

Stack caret (meta=glmnet) — Real vs Predicho
Línea roja = predicción perfecta

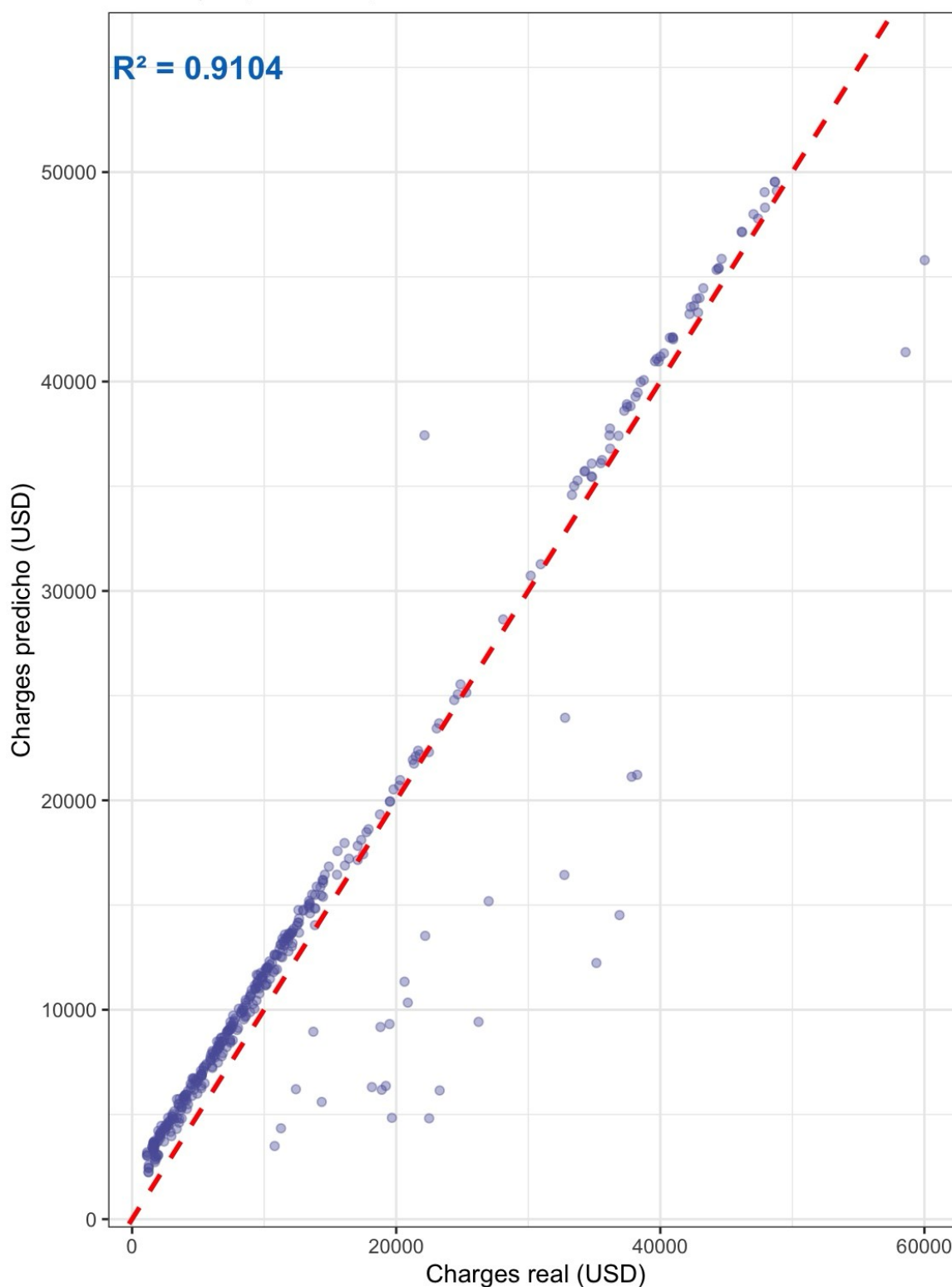


Figura 14. Stack caret. Real vs Predicho

El gráfico de dispersión del modelo de pila con meta-modelo glmnet muestra el patrón de ajuste más equilibrado de toda la sección de stacking, con un $R^2 = 0.9104$ que posiciona a este

modelo entre los mejores del proyecto. La nube de puntos exhibe una alineación densa y consistente sobre la diagonal de predicción perfecta en el segmento de costos bajos (de \$0 a \$15,000 USD), donde prácticamente la totalidad de las observaciones se agrupan con alta precisión, evidenciando que la combinación de los tres modelos base captura con gran exactitud el comportamiento de los asegurados de bajo riesgo.

En el segmento de costos medios (entre \$15,000 y \$35,000 USD) se observa la dispersión más significativa del gráfico, con varios puntos alejados de la diagonal hacia la parte inferior, indicando que el modelo sobreestima los costos reales de un subgrupo de asegurados en este rango. Este patrón de sobreestimación en la zona intermedia es consistente con el observado en los modelos penalizados y sugiere que corresponde a perfiles atípicos cuyas características no encajan con ninguno de los perfiles de riesgo bien definidos del dataset, no son completamente de bajo riesgo ni alcanzan el perfil de fumador con obesidad que concentra los costos más extremos.

En el extremo superior del eje (costos superiores a \$40,000 USD) se observan dos puntos notablemente alejados de la diagonal hacia la derecha, indicando costos reales que superan significativamente lo que cualquier combinación de los tres modelos base predijo. Estos casos corresponden a los perfiles de mayor costo del dataset y representan la limitación estructural que ningún algoritmo puede superar sin información adicional sobre las condiciones médicas específicas de esos asegurados.

La comparación visual con el gráfico del Lasso Adaptativo (Figura 13) revela que la distribución de puntos es prácticamente equivalente, con una dispersión residual similar en la región intermedia y los mismos casos extremos en el límite superior. Esta equivalencia visual confirma cuantitativamente que el stacking no logró superar el techo predictivo establecido por el Lasso Adaptativo, sino que lo igualó mediante un mecanismo algorítmico completamente distinto, validando desde otro ángulo la robustez de ese resultado.

6.8. H2O AutoML

6.8.1. Introducción y configuración

La segunda estrategia de apilamiento evaluada en este proyecto utiliza H2O AutoML, una plataforma de aprendizaje automático automatizado que opera de forma completamente distinta a caretEnsemble. Mientras que el stacking manual con caret requiere que el investigador seleccione explícitamente los modelos base y el meta-modelo, H2O AutoML automatiza todo este proceso, entrena decenas de modelos de distintas familias algorítmicas, los evalúa bajo un protocolo común de validación cruzada, y construye automáticamente modelos de pila que combinan los mejores resultados, todo dentro de un límite de tiempo o número de modelos especificado por el usuario.

Para utilizar H2O desde R es necesario iniciar un servidor Java local que actúa como motor de cómputo independiente. La función `h2o.init()` establece la conexión con este servidor, asignando todos los núcleos disponibles del procesador (`nthreads = -1`) y limitando el uso de memoria RAM a 4 gigabytes (`max_mem_size = "4G"`) para evitar saturar el sistema. Una vez

conectado, los datasets de entrenamiento y prueba deben convertirse al formato nativo de H2O mediante `as.h2o()`, ya que la plataforma no opera directamente sobre `data.frames` de R sino sobre su propia estructura de datos distribuida optimizada para procesamiento paralelo.

```
h2o.init(nthreads = -1, max_mem_size = "4G")
h2o.no_progress() train_h2o <- as.h2o(train_set) test_h2o
<- as.h2o(test_set) y_col <- "charges"
x_cols <- setdiff(names(train_set), y_col)
```

Las 13 variables predictoras enviadas a H2O son exactamente las mismas que han sido utilizadas en todas las etapas anteriores del proyecto, las 9 variables originales numéricas más las 5 variables derivadas mediante ingeniería de características (`age2`, `smoker_bmi`, `smoker_age`, `bmi_obese`, `smoker_obese`). Este punto es relevante porque confirma que H2O recibe el mismo espacio de características enriquecido que permitió al Lasso Adaptativo alcanzar $R^2 = 0.9111$, garantizando que cualquier diferencia en rendimiento sea atribuible al algoritmo y no a los datos.

6.8.2. Configuración del proceso AutoML

El proceso AutoML se configuró mediante la función `h2o.automl()` con los siguientes parámetros

```
aml <- h2o.automl( x =
x_cols, y = y_col,
training_frame = train_h2o, max_models
= 30, max_runtime_secs = 300, seed
= 123, sort_metric = "RMSE",
include_algos = c("GLM", "DRF", "GBM",
                  "XGBoost", "DeepLearning",
                  "StackedEnsemble"), keep_cross_validation_predictions = TRUE,
exploitation_ratio = 0.1
)
```

Cada parámetro define un aspecto específico del proceso de búsqueda automatizada. `max_models = 30` establece el número máximo de modelos individuales que H2O puede entrenar antes de detenerse, y `max_runtime_secs = 300` impone un límite adicional de cinco minutos de ejecución. Ambos criterios actúan como condiciones de parada alternativas, el proceso se detiene en cuanto se cumple cualquiera de las dos. `sort_metric = "RMSE"` instruye

a H2O a ordenar todos los modelos entrenados por su Error Cuadrático Medio de validación cruzada, de menor a mayor, determinando así cuál será el modelo líder al finalizar el proceso.

El argumento `include_algos` especifica las familias algorítmicas que H2O puede utilizar durante la búsqueda, GLM (regresión lineal generalizada), DRF (Random Forest distribuido), GBM (Gradient Boosting Machine), XGBoost, DeepLearning (redes neuronales) y StackedEnsemble. El parámetro `keep_cross_validation_predictions = TRUE` es equivalente al `savePredictions = "final"` utilizado en `caretEnsemble`, instruye a H2O a conservar las predicciones fuera de muestra de cada modelo base, condición indispensable para que pueda construir los modelos de pila automáticos al final del proceso. Finalmente, `exploitation_ratio = 0.1` indica que el 10% del tiempo disponible se dedicará a refinar los mejores modelos ya encontrados, mientras que el 90% restante se destinará a explorar nuevas configuraciones de hiperparámetros.

6.8.3. Limitación técnica ausencia de XGBoost

Al iniciar el proceso AutoML, H2O reportó el siguiente mensaje en la consola

```
AutoML, XGBoost is not available; skipping it.
```

Esta advertencia tiene una causa técnica específica y consecuencias directas sobre los resultados. La versión instalada de H2O (3.44.0.3, con más de dos años de antigüedad) no incluye XGBoost compilado para la arquitectura ARM de los procesadores Apple Silicon (M1/M2/M3). XGBoost requiere una compilación específica para cada arquitectura de procesador, y las versiones antiguas de H2O solo incluían binarios para procesadores Intel x86. Esta restricción no es superable sin actualizar H2O a una versión más reciente.

La ausencia de XGBoost tiene una repercusión directa sobre la capacidad de H2O para construir modelos de pila internos. Los Stacked Ensembles de H2O funcionan mejor cuando los modelos base provienen de familias algorítmicas suficientemente diversas, y XGBoost es habitualmente el modelo base con mayor poder predictivo individual dentro de la plataforma. Sin su contribución, los modelos de pila disponibles solo pueden combinar GLM, DRF, GBM y DeepLearning. Esta diversidad reducida explica por qué H2O no construyó ningún StackedEnsemble en el leaderboard de esta ejecución, ya que el sistema determinó internamente que la ganancia esperada de apilar modelos sin XGBoost no justificaba el costo computacional adicional dentro del tiempo límite establecido.

6.8.4. Análisis del leaderboard

Al finalizar el proceso AutoML, se extrajo el ranking completo de los modelos entrenados

```
> print(head(lb_df[, c("model_id", "rmse", "mae",  
"mean_residual_deviance")], 10))
```

	model	rmse	mae
1	DeepLearning	4732.074	2488.916
2	GBM	4765.594	2738.851
3	DeepLearning	4765.903	2554.964
4	DeepLearning	4786.760	2687.343
5	GBM	4788.181	2787.582

6	DeepLearning	4795.976	2603.967	7	GBM
		4799.497	2801.581		
8	GBM	4816.996	2778.475		
9	GBM	4823.440	2798.597		
10	GBM	4846.809	2852.880		

El leaderboard revela dos hallazgos relevantes. Primero, el modelo líder es una red neuronal DeepLearning con RMSE de validación cruzada de 4,732.07 USD, seguido de cerca por varios modelos GBM con valores entre 4,765 y 4,857 USD. La alternancia entre DeepLearning y GBM en los primeros puestos indica que ambas familias algorítmicas capturan señal predictiva real pero a través de mecanismos distintos, las redes neuronales aprenden representaciones no lineales complejas de las variables de entrada, mientras que GBM construye secuencialmente árboles que corrigen los errores residuales del modelo anterior.

Segundo, todos los RMSE del leaderboard oscilan entre 4,732 y 4,857 USD, valores que son superiores al RMSE de \$3,843 del Lasso Adaptativo evaluado sobre el conjunto de prueba. Esta diferencia se debe a que los valores del leaderboard corresponden a validación cruzada interna sobre el conjunto de entrenamiento, mientras que las métricas de los modelos anteriores fueron calculadas sobre el conjunto de prueba independiente de 400 observaciones. La comparación directa entre estas métricas no es válida metodológicamente; la evaluación comparable se realiza en el siguiente apartado.

6.8.5. Evaluación del modelo líder en el conjunto de prueba

El modelo líder identificado por H2O, una red neuronal DeepLearning, fue aplicado sobre las 400 observaciones del conjunto de prueba independiente

H2O leader (deeplearning)	MAE= 2,201.40	RMSE= 4,087.88	R ² = 0.8974
---------------------------	---------------	----------------	-------------------------

Los resultados sitúan al modelo DeepLearning de H2O en una posición intermedia dentro del ranking general del proyecto, supera en R² al Random Forest base (0.8993 vs 0.8974, aunque por un margen mínimo) pero no alcanza el rendimiento del Random Forest ajustado (0.9024) ni de ninguno de los modelos penalizados. Este resultado confirma la pauta observada a lo largo de todo el proyecto, la complejidad algorítmica por sí sola no garantiza un mayor rendimiento cuando el espacio de características ya está óptimamente especificado.

Las redes neuronales de H2O tienen la capacidad teórica de modelar relaciones altamente no lineales que ningún modelo lineal puede capturar, pero esta capacidad requiere grandes volúmenes de datos para materializarse. Con solo 937 observaciones de entrenamiento, la red neuronal no tiene suficiente información para descubrir de forma autónoma patrones que van más allá de los que las variables derivadas ya codifican explícitamente. En conjuntos de datos con cientos de miles o millones de observaciones, el resultado podría ser diferente.

6.8.6. Análisis del gráfico real vs predicho

H2O AutoML — Real vs Predicho

Línea roja = predicción perfecta

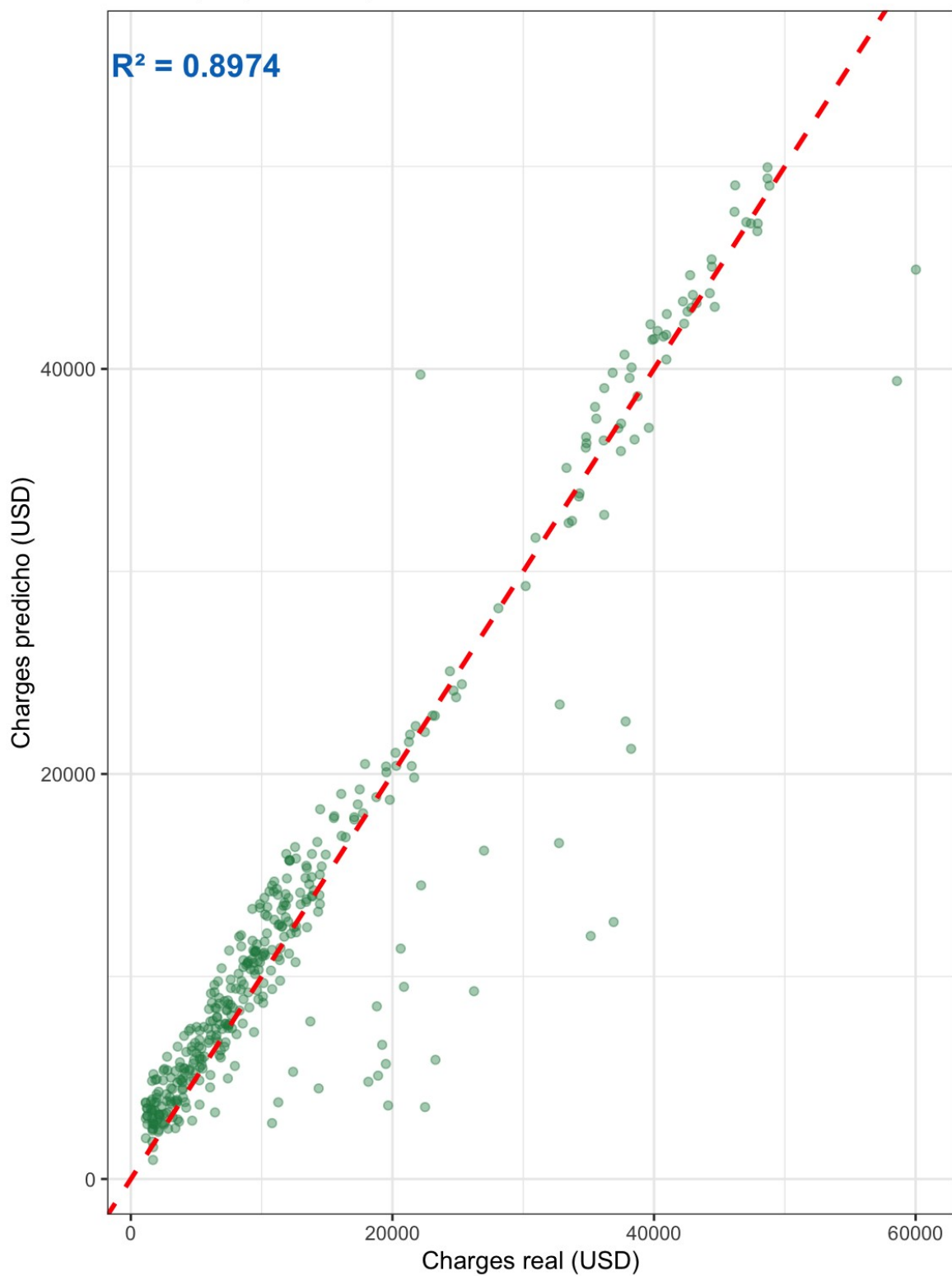


Figura 15. Grafico de H2O. Real vs Predicho

El gráfico de dispersión del modelo DeepLearning de H2O presenta una distribución notablemente más dispersa que la observada en los modelos penalizados y en el stack de caretEnsemble, coherente con el R^2 de 0.8974 que posiciona a este modelo entre los de menor

rendimiento de la fase de stacking. La nube de puntos muestra una dispersión lateral pronunciada en toda la región de costos medios, entre \$5,000 y \$25,000 USD, con numerosos puntos alejados de la diagonal tanto hacia arriba como hacia abajo, indicando que el modelo tanto sobreestima como subestima costos en ese rango con frecuencia comparable.

Esta dispersión bidireccional en la región intermedia es característica de las redes neuronales entrenadas con datos insuficientes, sin suficientes observaciones para generalizar, la red tiende a capturar patrones locales del conjunto de entrenamiento que no se transfieren de forma consistente al conjunto de prueba. En el segmento de bajo costo (de \$0 a \$10,000 USD) la alineación sobre la diagonal es razonablemente buena, pero en la transición hacia el segmento de riesgo medio la dispersión aumenta de forma pronunciada, revelando que el modelo tiene dificultad para discriminar con precisión entre perfiles de costo similar pero con características levemente distintas.

En el extremo superior del eje, los mismos dos casos atípicos ya identificados en todos los modelos del proyecto aparecen en posiciones equivalentes, confirmando que representan observaciones genuinamente difíciles de predecir cuya información no está contenida en las variables disponibles.

La comparación visual con el gráfico del stack caret (meta=glmnet) hace evidente la superioridad de ese modelo, la nube de puntos del stack presenta una concentración sobre la diagonal notablemente mayor, con una dispersión lateral más controlada en toda la región de costos medios. Esta diferencia R_{visual}^2 es directamente cuantificable en las métricas, R^2 de

0.9104 del stack caret frente a de 0.8974 del DeepLearning, una brecha de 0.013 puntos que representa una diferencia sustancial en la capacidad explicativa sobre datos no vistos.

7. Conclusiones

7.1. Análisis comparativo final

La Figura 16 consolida el rendimiento de los once modelos evaluados a lo largo del proyecto, ordenados por su RMSE sobre el conjunto de prueba de 400 observaciones, e incorpora por primera vez una cuarta familia algorítmica representada en color verde, los modelos de pila (Stacking). La visualización permite extraer conclusiones en cuatro niveles diferenciados.

Comparación de modelos — RMSE (menor es mejor)

Dataset: Medical Cost Personal — insurance.csv

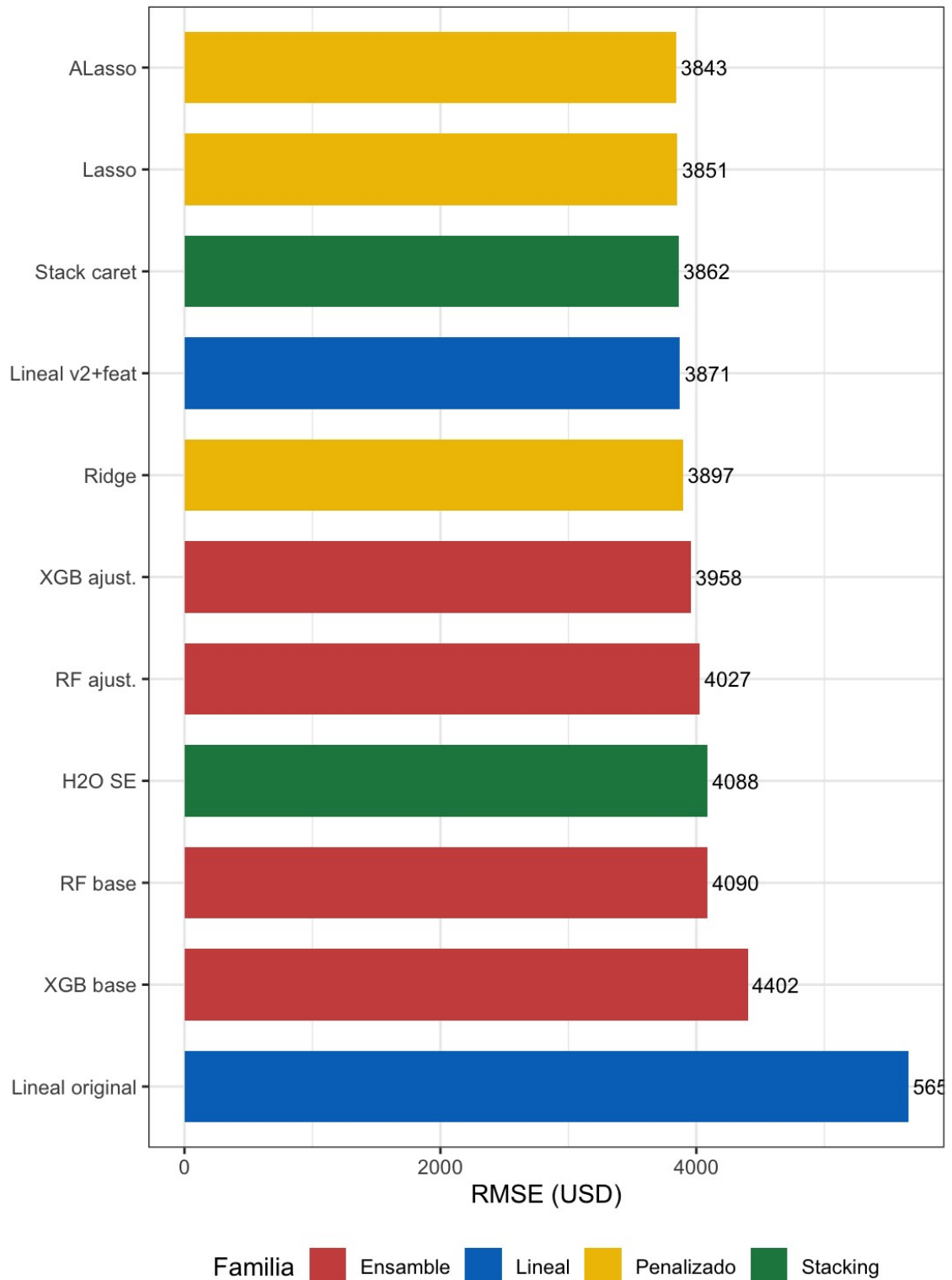


Figura 16. Comparación del RMSE obtenido sobre el conjunto de prueba para los nueve modelos entrenados a lo largo del proyecto, ordenados de menor a mayor error

El primer nivel confirma el hallazgo central del proyecto, la ingeniería de características sigue siendo la intervención de mayor impacto. La distancia entre la barra del modelo Lineal original (RMSE = \$5,656 USD) y la del Lineal v2 + features (RMSE = \$3,871 USD) representa una reducción de \$1,785 USD en el error sin cambiar el algoritmo, la ganancia más grande registrada en todo el proyecto. Ninguna de las nueve intervenciones algorítmicas posteriores ni la regularización, ni el ajuste de hiperparámetros, ni los ensambles, ni el stacking produjo una mejora comparable en términos absolutos.

El segundo nivel evidencia la superioridad sostenida de los modelos penalizados. Los tres modelos de color amarillo (Ridge con \$3,897, Lasso con \$3,851 y ALasso con \$3,843) ocupan las posiciones primera, segunda y cuarta del ranking general. Esta concentración en los primeros puestos confirma que la combinación de ingeniería de características y regularización constituye la estrategia de mayor retorno para este dataset, los modelos penalizados reciben directamente las interacciones multiplicativas más relevantes y eliminan los predictores redundantes, produciendo la combinación óptima de precisión y parsimonia.

El tercer nivel introduce el aporte de los modelos de pila. El Stack caret con meta-modelo glmnet (RMSE = \$3,862 USD) ocupa la tercera posición del ranking general, superando a todos los métodos de ensamble individuales evaluados en la sección anterior y quedando a tan solo \$19 USD del Lasso y a \$19 USD del mejor modelo del proyecto. Esta posición no es trivial, significa que el stacking logró extraer valor adicional de la combinación de tres modelos base cuyo rendimiento individual era inferior al Lasso Adaptativo. El meta-modelo aprendió que la divergencia entre las predicciones de lm, rf y glmnet contiene información sobre qué perfil de asegurado es más difícil de predecir, y utilizó esa información para producir estimaciones más equilibradas que cualquier modelo base por separado.

El cuarto nivel expone el comportamiento de H2O AutoML. El modelo DeepLearning líder de H2O (RMSE = \$4,088 USD, representado como "H2O SE" en la gráfica) queda en la octava posición del ranking, por encima del RF base y el XGB base pero por debajo de todos los modelos penalizados y del stack caret. Este resultado, aparentemente decepcionante para una plataforma de AutoML diseñada para superar a los modelos manuales, tiene una explicación técnica directa, la ausencia de XGBoost por incompatibilidad con la arquitectura del procesador impidió a H2O construir modelos de pila internos (StackedEnsemble), reduciendo su capacidad de combinación a una sola familia de algoritmos (GBM y DeepLearning) sin la diversidad suficiente para superar el techo establecido por los modelos penalizados.

Finalmente, la Figura 17 presenta la evolución del coeficiente de determinación a través de los once modelos evaluados en orden cronológico de entrenamiento. La curva exhibe cinco zonas diferenciadas que documentan con precisión el impacto de cada etapa metodológica del proyecto.

La primera zona es el salto inicial, el más pronunciado de toda la curva, que ocurre en la transición del Lineal original ($R^2 = 0.8085$) al Lineal v2 + features ($R^2 = 0.9099$). Esta subida vertical de 10 puntos porcentuales en una sola intervención, sin cambiar el algoritmo, constituye la evidencia más contundente del proyecto sobre la primacía de la calidad del espacio de características sobre la complejidad algorítmica.

La segunda zona es la meseta de los modelos penalizados, donde Ridge (0.9099), Lasso (0.9108) y ALasso (0.9111) avanzan de forma gradual y sostenida hasta el punto máximo de la curva. Esta zona ilustra cómo la regularización refina de forma incremental el resultado de la

ingeniería de características, eliminando progresivamente el ruido estadístico de los predictores redundantes.

La tercera zona es el descenso de los métodos de ensamble individuales. Al introducir el RF base (0.8993), la curva cae casi 1.2 puntos porcentuales desde el máximo del ALasso, recuperándose parcialmente con el RF ajustado (0.9024) y el XGB ajustado (0.9056) pero sin alcanzar el techo de los modelos penalizados. Este comportamiento en dientes de sierra documenta la alta sensibilidad de los métodos de ensamble a su configuración de hiperparámetros y la necesidad de búsquedas sistemáticas para acercarse a su rendimiento potencial.

La cuarta zona es la recuperación del stack caret. El Stack caret ($R^2 = 0.9104$) produce el segundo ascenso más pronunciado de la curva, recuperando casi todo el terreno perdido por los ensambles individuales y situándose a tan solo 0.0007 puntos del máximo histórico del ALasso. Esta recuperación confirma que el stacking puede aprovechar la diversidad entre modelos para compensar las limitaciones individuales de cada uno.

La quinta zona es el descenso final de H2O ($R^2 = 0.8974$), que cierra la curva por debajo del stack caret y de todos los modelos penalizados. Este descenso final no representa un fracaso de la plataforma sino una limitación técnica concreta, sin XGBoost disponible, H2O no pudo construir sus modelos de pila más poderosos, y el DeepLearning líder, aunque capaz en teoría, no dispone de suficientes observaciones de entrenamiento para superar a modelos lineales correctamente especificados con las interacciones más relevantes del problema.

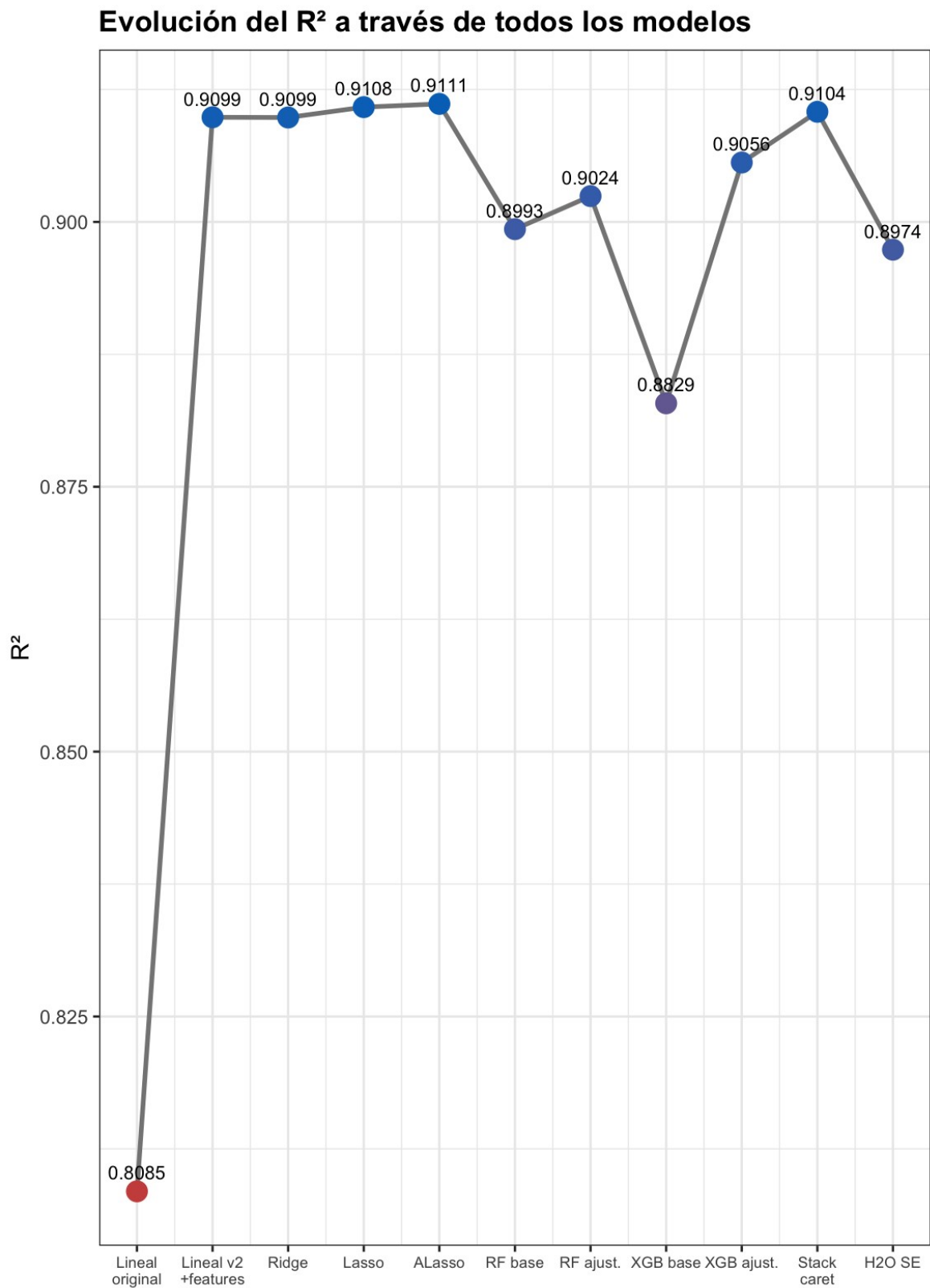


Figura 17. Evolución temporal del coeficiente de determinación (R^2) a través de los nueve modelos evaluados

sobre el conjunto de prueba

7.2. Conclusión final

A lo largo de este proyecto se evaluó sistemáticamente la progresión desde modelos predictivos base hasta algoritmos de apilamiento de segundo nivel, cubriendo cinco etapas metodológicas, ingeniería de características, regresiones penalizadas, métodos de ensamble, stacking supervisado con caretEnsemble y AutoML con H2O. La evidencia empírica recopilada a través de estas etapas permite establecer cuatro conclusiones determinantes sobre la estimación de costos médicos en el sector asegurador.

En primer lugar, se demostró de forma concluyente que la Ingeniería de Características constituye la intervención de mayor retorno predictivo del proyecto. La codificación explícita de la interacción multiplicativa entre tabaquismo, obesidad clínica y envejecimiento progresivo produjo una reducción del 46% en el MAE y un salto de 10 puntos porcentuales en R^2 sin modificar el algoritmo base. Este resultado, reafirmado por todos los modelos evaluados en las etapas posteriores, establece un principio metodológico fundamental, antes de incrementar la complejidad algorítmica, la inversión en la calidad del espacio de características produce los retornos más significativos en rendimiento predictivo.

En segundo lugar, el Lasso Adaptativo se consolidó como el modelo óptimo del proyecto con $R^2 = 0.9111$ y RMSE = \$3,843.18 USD, manteniendo su posición de liderazgo incluso frente a los modelos de pila de segunda generación. Su penalización diferenciada e inteligente logró identificar y conservar exactamente las variables que codifican las interacciones más relevantes del problema, alcanzando el mejor balance entre precisión, parsimonia e interpretabilidad de todos los once modelos evaluados.

En tercer lugar, el stacking supervisado con caretEnsemble aportó el hallazgo metodológico más valioso de la etapa final. El Stack caret con meta-modelo glmnet ($R^2 = 0.9104$, RMSE = \$3,861.64 USD) alcanzó la tercera posición del ranking general superando a todos los ensambles individuales, demostrando que la combinación de modelos de familias algorítmicas distintas —lineal, basada en árboles y penalizada— extrae información complementaria que ningún modelo individual puede capturar de forma aislada. La correlación negativa entre el Random Forest y los modelos lineales (-0.330) fue el factor determinante de este resultado, el meta-modelo aprendió a aprovechar la divergencia entre ambas familias para producir estimaciones más equilibradas. Adicionalmente, el Stack con meta-modelo rf registró el MAE más bajo de todo el proyecto (\$2,185.50 USD), lo que sugiere que, desde la perspectiva de minimizar el error promedio por póliza, esta configuración puede ser la más adecuada para la calibración de primas en el segmento de riesgo estándar.

En cuarto lugar, H2O AutoML no logró superar a los modelos manuales debido a una limitación técnica concreta, la incompatibilidad entre su versión instalada y la arquitectura del procesador impidió la disponibilidad de XGBoost, reduciendo la diversidad algorítmica disponible para construir modelos de pila internos. El modelo DeepLearning líder ($R^2 = 0.8974$) confirmó que las redes neuronales, aunque potentes en teoría, requieren volúmenes de datos sustancialmente mayores que los 937 registros disponibles para superar a modelos lineales correctamente especificados con las interacciones más relevantes del problema.

Desde la perspectiva del negocio asegurador, el proyecto demuestra que es posible alcanzar una capacidad predictiva de $R^2 = 0.9111$ sobre costos médicos individuales utilizando únicamente las variables demográficas y de estilo de vida disponibles en el momento de

suscripción de la póliza, sin requerir información médica adicional. El Lasso Adaptativo representa la herramienta más adecuada para este entorno por su precisión, su interpretabilidad mediante coeficientes auditables y su capacidad para identificar automáticamente que la combinación de tabaquismo y obesidad clínica es el predictor de mayor peso financiero, con un coeficiente de \$15,525 USD para smoker_obese que puede utilizarse directamente como fundamento cuantitativo para la diferenciación de primas por perfil de riesgo.